



kubernetes



KubeVirt



목차

쿠버네티스 가상화



목차



목차



과정개요

쿠버네티스 가상화

강사

랩



소개

이 교육은 다음과 같은 목표를 가지고 있음.

1. 쿠버네티스 기반으로 PaaS 가상화 시스템 구축 및 운영
2. 기존에 사용중인 쿠버네티스 플랫폼에 가상화 기능 통합
3. 기존에 사용하던 VMWare, OpenStack, RHV를 쿠버네티스 플랫폼으로 마이그레이션
4. 가상머신 기반 빠른 서비스 배포를 고려중인 서비스 아키텍처 회사



대상

01

쿠버네티스 운영 경험이
있는 사용자

기존 가상머신 환경을
마이그레이션이 필요한
인프라 엔지니어

02

기본적인 리눅스의 표준
가상화 지식이 있는 사용자

Libvirt

Guestfish-tools

oz, dib

03

쿠버네티스를 통한 가상머신
운영을 원하는 서비스 아키
텍처 혹은 인프라 엔지니어



과정개요

현재 CNCF 및 컨테이너 기반 플랫폼의 표준으로 자리 잡은 쿠버네티스(Kubernetes)에 대해 학습하는 과정. 본 교육에서는 쿠버네티스 설치, 구성, 운영을 다루며, OCI 도구와 쿠버네티스의 관계도 함께 살펴본다.

과정은 3일/5일에 따라서 구성이 달라진다. 내용은 다음과 같이 차이가 있다.

길이	설명



강사

강사



강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.



LAB

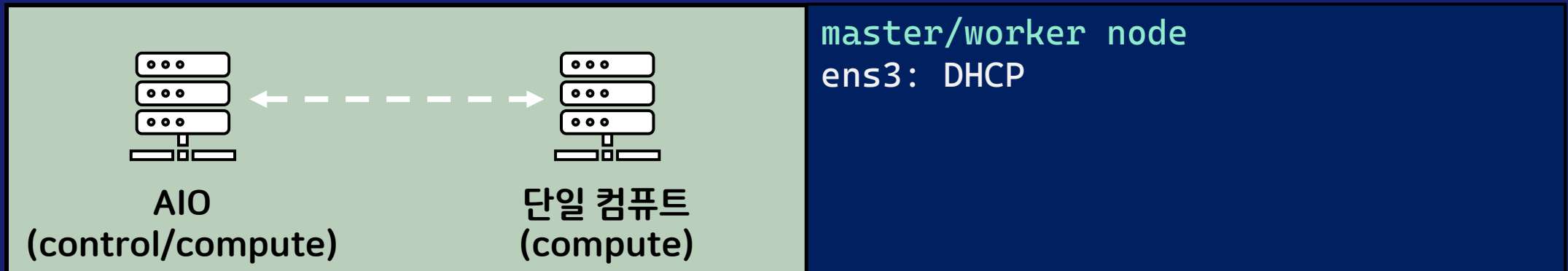
설명



가상머신 구성

기본과정이기 때문에 총 2대의 가상머신을 사용합니다. 컨테이너 및 가상머신 혼합 및 혼용 환경으로 운영 및 CICD는 통합 CICD과정에서 다룰 예정입니다.

네트워크는 ens3만 사용하며, 별도로 추가가 되는 네트워크는 없습니다.



소개

누가 만들었는가?

쿠버네티스 가상화



누가 만들었는가?



Kube-Virt의 탄생 배경

KubeVirt 쿠버네티스 환경에서 가상머신(VM)을 네이티브하게 실행하기 위해 시작된 오픈소스 프로젝트이다.
컨테이너 중심의 Kubernetes 환경에서 기존 VM 기반 워크로드를 함께 운영할 필요성에서 출발하였다.

주요 주도 및 참여 기업

Red Hat

- 프로젝트의 핵심 설계 및 초기 개발을 주도
- Kubernetes 기반 하이브리드 가상화 전략의 일환

IBM, Intel, NVIDIA 등

- 엔터프라이즈 및 하드웨어 벤더들이 커뮤니티 형태로 참여

“KubeVirt는 Red Hat을 중심으로 한 오픈소스 커뮤니티가 Kubernetes를 컨테이너와 VM을 아우르는 플랫폼으로 확장하기 위해 만든 프로젝트이다.”



Kube-Virt의 탄생 배경

현재 많은 가상화 기업들이 쿠버네티스와 통합을 위해서 기존에 사용하던 가상머신 플랫폼을 포기하고 kube-virt로 통합 혹은 마이그레이션 작업을 많이 이루고 있다.

kube-virt를 사용하는 대표적이 미들웨어 솔루션은 다음과 같다.

1. Redhat OpenShift Virtualization
2. SUSE-VIRT(AKA Harvester)
3. VMWARE Tanzu 현재는 중지, kube-virt 지원하지 않음.



쿠버네티스 가상화 설명

아키텍처

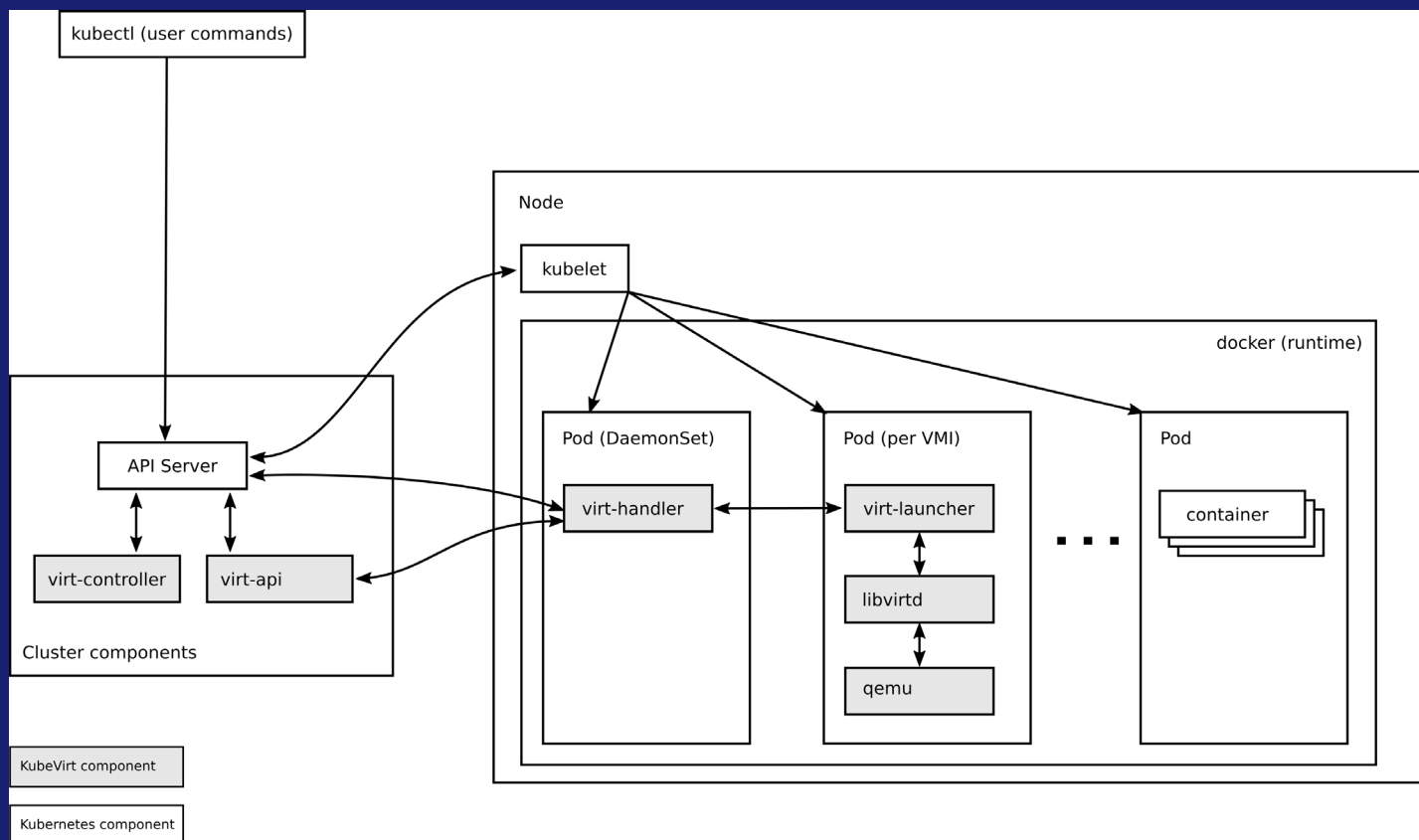
탄생



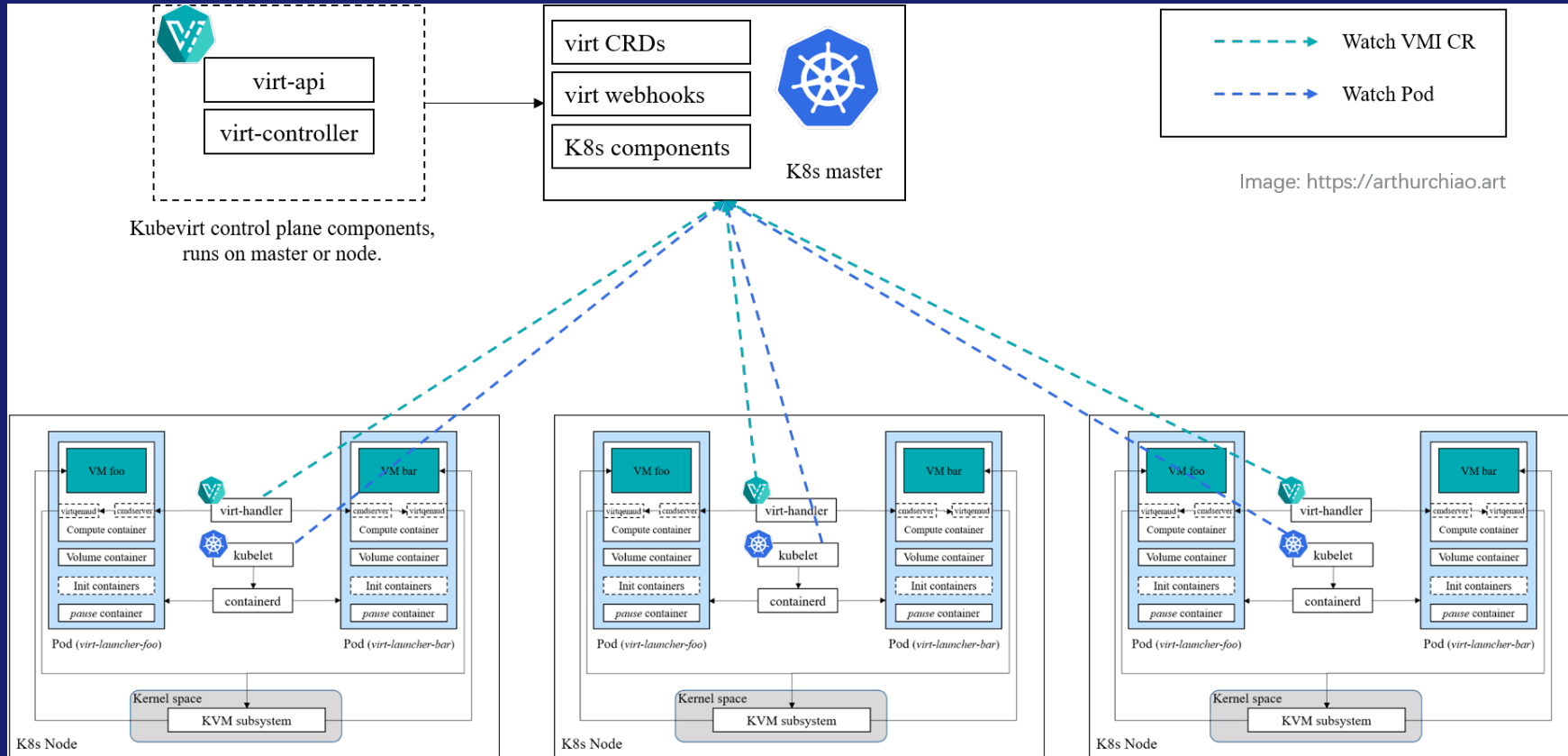
아키텍처



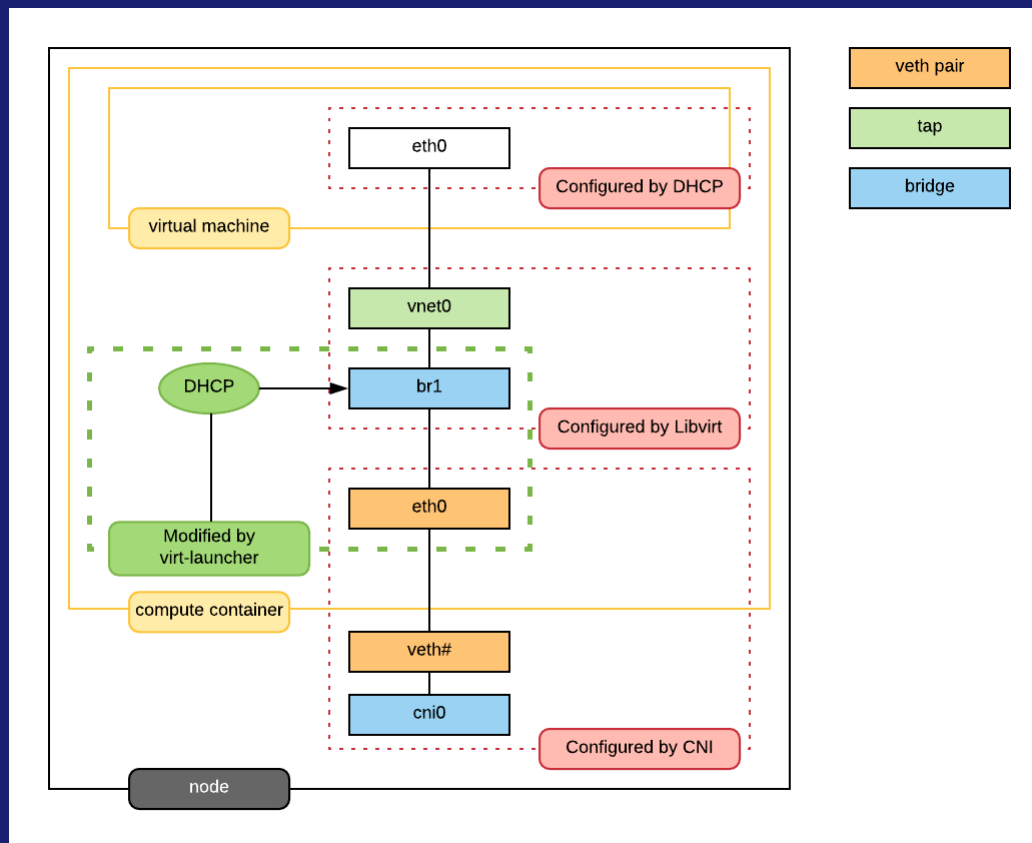
아키텍처 1



아키텍처 2



가상머신 구성 구조



VM 생성 흐름

① 사용자 요청

- 사용자가 VirtualMachine 또는 VirtualMachineInstance(VMI) 생성
- Kubernetes API로 선언적 요청

② 컨트롤 플레인 처리

- virt-controller가 VMI를 감지
- VM 실행을 위한 Pod 스펙 자동 생성



VM 생성 흐름

③ Pod 생성

- Kubernetes가 특수 Pod(virt-launcher Pod) 생성
- 이 Pod 안에:
 - QEMU
 - libvirt
 - VM 디스크(CSI)
 - VM 네트워크(CNI)

④ VM 실행

- Pod 내부에서 QEMU 프로세스로 VM 부팅(이 때, 두 개의 컨테이너가 Pod에서 동작)
- Kubernetes 입장에서는 Pod
- 사용자 입장에서는 VM 컨테이너, libvirt 컨테이너



흐름 아스키 도식

위의 내용을 아스키 도식으로 다음과 같이 구성이 가능하다.

```
[ User/kubectl ]
|
v
[ Kubernetes API Server ]
|
v
[ VirtualMachine/VMI (CRD) ]
|
v
[ virt-controller ]
  (VM → Pod 변환)
|
v
[ virt-launcher Pod ] ← Kubernetes가 인식하는 실행 단위
|
v
[ QEMU+libvirt ]
|
v
[ Virtual Machine(Guest OS) ]
```



설명

쿠버네티스 가상화



쿠버네티스 가상화의 개념

Kubernetes 가상화란

전통적인 하이퍼바이저 중심의 가상머신(VM) 방식이 아니라, 컨테이너(Container) 를 기본 실행 단위로 사용하는
경량 가상화 모델이다.

운영체제 전체를 가상화하지 않고, 커널을 공유하면서 애플리케이션 단위로 격리한다.



기존 가상화와의 차이

VM 가상화

하드웨어 → 하이퍼바이저 → Guest OS → 애플리케이션

쿠버네티스 가상화

하드웨어 → OS 커널 → 컨테이너 → 애플리케이션



핵심 특징

1. 빠른 기동 속도
2. 높은 리소스 밀도
3. 선언적(Declarative) 인프라 운영



전체 구조 개요

Control Plane

- API Server, Scheduler, Controller Manager
- 클러스터 상태와 워크로드를 제어

Worker Node

- 컨테이너 런타임(Containerd, CRI-O 등)
- 실제 애플리케이션(Pod)이 실행되는 영역



가상화 관점에서의 역할

- Pod는 가상 실행 단위
- Namespace는 논리적 격리 공간
- CNI는 가상 네트워크
- CSI는 가상 스토리지

결과적으로 컴퓨트, 네트워크, 스토리지가 모두 소프트웨어 정의(SDI) 형태로 가상화 됨.



왜 VM 가상화가 필요한가?

- 기존 레거시 애플리케이션
- Windows, 커널 의존 서비스
- 컨테이너 전환이 어려운 워크로드



KubeVirt 개요

- 쿠버네티스 위에서 VM을 Pod처럼 실행
- 컨테이너와 VM을 동일한 API로 관리
- Kubernetes 스케줄링, 네트워크, 스토리지를 그대로 활용



하이브리드 가상화의 장점

- 컨테이너+VM 동시 운영
- 점진적 마이그레이션 가능
- 단일 플랫폼으로 운영 복잡도 감소



설치

쿠버네티스 가상화



설치 조건

설치를 위해서 다음과 같은 조건이 필요하다.

1. 최소 한 개의 쿠버네티스 클러스터 혹은 노드
2. 가상화 기능을 지원하는 하드웨어 혹은 가상머신
3. 쿠버네티스 버전 1.30이상 버전 권장
4. 한 개 이상의 스토리지 클래스 권장



설치 시작

이미 쿠버네티스 클러스터가 존재하는 조건으로 다음과 같은 순서로 매니페스트 설치를 진행한다.

```
export KUBEVIRT_VERSION=$(curl -s  
https://api.github.com/repos/kubevirt/kubevirt/releases/latest | grep  
tag_name | cut -d '"' -f 4)  
export KUBEVIRT_VERSION=v1.7.0
```



오퍼레이터 구성

kube-virt는 오퍼레이터 기반으로 설치가 된다. 아래처럼 매니 페스트를 등록한다.

```
# kube-virt operator
```

```
kubectl apply -f
```

```
https://github.com/kubevirt/kubevirt/releases/download/${KUBEVIRT_VERSION}/  
kubevirt-operator.yaml
```

```
# kube-virt CR 구성
```

```
kubectl apply -f
```

```
https://github.com/kubevirt/kubevirt/releases/download/${KUBEVIRT_VERSION}/  
kubevirt-cr.yaml
```



오퍼레이터 설치 확인

올바르게 설치가 완료가 되면 아래 명령어로 확인한다.

```
kubectl -n kubevirt get all  
kubectl get -n kubevirt kubevirt  
kubectl get -n kubevirt pod -w
```



확장 명령어 설치 및 가상화 지원 확인

쿠버네티스에서 가상머신 구성/생성/관리는 아래 명령어로 가능하다. 간단한 조회는 여전히 kubectl 명령어로 가능하다.

```
curl -L -o virtctl
https://github.com/kubevirt/kubevirt/releases/download/${KUBEVIRT_VERSION}/
virtctl-${KUBEVIRT_VERSION}-linux-amd64chmod +x virtctlsudo mv virtctl
/usr/local/bin/
kubectl completion > /etc/profile.d/virtctl.sh
source /etc/profile.d/virtctl.sh
```

```
lsmod | grep kvm
modprobe kvm
modprobe kvm_intel      # Intel
modprobe kvm_amd        # AMD
```



CDI(Containerized Data Importer) 설치

컨테이너 이미지를 관리를 위한 이미지 데이터 가져오기를 설치한다.

```
export CDI_VERSION=$(curl -s  
https://api.github.com/repos/kubevirt/containerized-data-  
importer/releases/latest | grep tag_name | cut -d '"' -f 4)
```

```
kubectl apply -f https://github.com/kubevirt/containerized-data-  
importer/releases/download/${CDI_VERSION}/cdi-operator.yaml
```

```
kubectl apply -f https://github.com/kubevirt/containerized-data-  
importer/releases/download/${CDI_VERSION}/cdi-cr.yaml
```



CDI(Containerized Data Importer) 확인

올바르게 설치가 되었는지 확인 및 이미지 업로드 한다.

```
kubectl -n cdi get pods  
virtctl image-upload --help
```

```
mkdir -p ~/kubevirt-imagescd ~/kubevirt-images  
curl -LO https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-  
disk.img  
ls -lh cirros-0.6.2-x86_64-disk.img
```



CDI(Containerized Data Importer) 확인

올바르게 설치가 되었는지 확인 및 이미지 업로드 한다.

```
virtctl image-upload dv cirros-dv --size=2Gi \  
--image-path=cirros-0.6.2-x86_64-disk.img --namespace=cdi \  
--storage-class=standard \  
--uploadproxy-url=https://cdi-uploadproxy.default.svc.cluster.local:443 \  
--insecure
```

```
kubectl get sc  
kubectl get dv  
kubectl get pvc
```



CDI(Containerized Data Importer) 확인

올바르게 DNS질의가 되지 않는 경우 ClusterIP로 접근한다.

```
kubectl get -n cdi svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
cdi-uploadproxy	ClusterIP	172.20.84.96	<none>	443/TCP

```
virtctl image-upload dv cirros-dv --size=2Gi \  
--image-path=cirros-0.6.2-x86_64-disk.img --namespace=cdi \  
--storage-class=standard \  
--uploadproxy-url=https://172.20.84.96:443 \  
--insecure
```



스토리지 프로파일 설명(sp)

스토리지 프로파일은 VM 생성 시, 사용하는 디스크 자원이다. 이 자원을 통해서 가상머신이 사용할 디스크를 생성한다. 스토리지 프로파일은 StorageClass, PersistentVolumeClaim이 올바르게 구성이 되어 있어야 사용이 가능하다.



스토리지 프로파일 생성(sp)

테스트용도 가상머신이 생성 될 스토리지 프로파일을 생성해야 한다. 없는 경우 올바르게 가상머신이 생성이 되지 않는다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: StorageProfile
metadata:
  name: nfs-client
spec:
  claimPropertySets:
  - accessModes:
    - ReadWriteMany
    volumeMode: Filesystem
```

EOF



스토리지 클래스 변경(sc)

가상머신이 NFS 기반으로 생성이 되는 경우, RWX로 구성을 권장한다. 아래와 같이 변경을 권장한다.

```
kubectl patch storageprofile nfs-client --type='merge' -p \
'{"spec":{"claimPropertySets":[{"accessModes":["ReadWriteOnce"],"volumeMode":"Filesystem"}]}}'
```



가상머신 생성 1

간단하게 가상머신을 생성 후 올바르게 구성이 되었는지 확인한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cirros-vm
spec:
  running: true
  template:
    spec:
      domain:
        cpu:
          cores: 1
```



가상머신 생성 2

```
resources:
  requests:
    memory: 256Mi
devices:
  disks:
    - name: rootdisk
      disk:
        bus: virtio
volumes:
- name: rootdisk
  persistentVolumeClaim:
    claimName: cirros-dv
```

EOF



가상머신 생성 확인

올바르게 생성이 되었는지 확인한다.

```
kubectl get vmi
```

```
cirros-vm    Running
```

```
kubectl get pvc
```

```
cirros-dv          Bound      pvc-9f2ac056-2624-4e2a-92b6-8fff8a34234d
2276332667    RWX          nfs-client    <unset>          3h35m
```

```
virtctl console cirros-vm
```

```
login: cirros
```

```
password: gocubsgo
```



기본 명령어 학습

기본 자원 다루기



시작 전...

쿠버네티스 가상화는 두 가지 명령어를 사용한다.

1. 쿠버네티스 기본 명령어 kubectl
2. 쿠버네티스 가상화 명령어 virtctl

이 과정에서는 이 두 개 명령어가 지원하는 기능에 대해서 확인한다. 명령어 활용 부분은 뒤에 가상머신/스토리지/네트워크 부분에서 다루기 다시 다룬다.



명령어 구별

쿠버네티스 가상화를 사용하면 두 가지 명령어를 통해서 관리가 필요하다.

1. kubectl
2. virtctl

kubectl는 본래 용도처럼 쿠버네티스에서 구성 및 관리된 자원을 관리하는 명령어이다. 일반적으로 사용하는 PV/PVC/SC 그리고 VM/VMI/DC와 같은 자원으로 분리가 된다.

kubectl를 통해서 아래와 API 자원을 분리하여 확인이 가능하다.



API 자원 확인

API에서 다음과 같이 자원이 분류 및 확인이 가능하다.

```
kubectl api-resources | grep kubevirt.io | awk 'BEGIN { printf "%-35s %-10s\n", "FULL", "SHORT" } { printf "%-35s %-10s\n", $1, $2 }'
```

FULL	SHORT
cdiconfigs	cdi.kubevirt.io/v1beta1
cdis	cdi,cdis
dataimportcrons	dic,dics
datasources	das
datavolumes	dv,dvs
objecttransfers	ot,ots



가상머신 확인 1

제일 많이 사용하는 명령어는 바로 가상머신이 올바르게 동작하는가? 이다. 가상머신 동작상태를 확인하기 위해서 아래와 같이 명령어로 확인이 가능하다. 다만, 두 가지 자원에 대해서 구별해야 한다.

1. `VirtualMachine (vm)`

2. `VirtualMachineInstance (vmi)`

두 개는 비슷하게 보이지만, 다른 형식으로 자원을 관리 및 구별하고 있다.

`VirtualMachine`은 가상머신이 미사용중인 상태이어도 화면에 출력이 된다. 즉, 인스턴스 목록과 비슷하다. 이 안에는 가상머신이 실행 시 사용하는 일종의 가상머신 사양을 가지고 있다.

```
kind: VirtualMachine
```

```
spec:
```

```
  running: true
```



가상머신 확인 2

VirtualMachineInstance (vmi)는 실제로 동작중인 가상머신 자원을 말한다. 이러한 이유로 뒤에 Instance라고 단어가 붙어있다.

역할

- 실제 QEMU/KVM 프로세스
- 노드에 스케줄링됨
- CPU/메모리/네트워크를 실제로 소비

```
kind: VirtualMachineInstance
status:
  phase: Running
```

특징

- VM이 켜지면 자동 생성
- VM을 끄면 자동 삭제
- Pod와 매우 유사한 라이프사이클



가상머신 확인 3

위의 내용을 정리하면 다음과 같다.

VirtualMachine (vm)

└ spec.running: true



VirtualMachineInstance (vmi) 생성



노드에 스케줄링



QEMU/KVM(Libvirt)



가상머신 확인 4

아래와 같이 기능 확인이 가능하다. 강제로 patch를 통해서 true → false으로 변경한다.

```
kubectl get vm
```

NAME	AGE	STATUS	READY
cirros-vm	3h54m	Running	True

```
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME	READY
cirros-vm	3h54m	Running	172.16.236.151	kubevirt.example.com	True

```
kubectl patch vm cirros-vm --type merge -p '{"spec":{"running":false}}'
```

```
kubectl get vm
```

NAME	AGE	STATUS	READY
cirros-vm	3h55m	Stopped	False

```
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME	READY
cirros-vm	3h55m	Succeeded		kubevirt.example.com	False



가상머신 확인 마지막

이 기능을 오픈스택 및 쿠버네티스와 비교하면 다음과 같다.

개념	KubeVirt	Kubernetes	OpenStack
설계/의도	VM	Deployment	Server
실행 인스턴스	VMI	Pod	Instance
전원 제어	VM	Replica	nova start/stop



DV 1

DV, DataVolume은 가상머신 생성 시, 사용할 볼륨/디스크 생성한다. 오픈스택과 비교하면 Glance에서 Cinder로 이미지 전환하는 과정이다.

```
kubectl get dv
```

NAME	PHASE	PROGRESS	RESTARTS	AGE
cirros-dv	Succeeded	100.0%	0	6m



DV 2

DV, DataVolume

가상머신 생성 시, 사용할 볼륨/디스크 생성한다. 오픈스택과 비교하면 Glance에서 Cinder로 이미지 전환하는 과정이다.

```
kubectl get dv
```

NAME	PHASE	PROGRESS	RESTARTS	AGE
cirros-dv	Succeeded	100.0%	0	6m



PVC

PVC

DV에서 구성된 데이터를 쿠버네티스로 가져오면, 해당 정보는 Pod에서 사용이 가능하도록 PVC정보로 변경이 된다.

OpenStack 비교

- PVC=Cinder Volume (또는 Nova에서 붙이는 Block Device)
- StorageClass=Cinder backend 타입(NFS/Ceph/LVM 같은 스토리지 정책)

```
kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY
cirros-dv	Bound	pvc-7f1b2d86-5d1b-4bde-a0f0-2d2c7a3c9f1a	2Gi



DATAVOLUME(dv)

DataVolume

올바르게 구성이 되면, 데이터 볼륨을 통해서 인스턴스가 올바르게 구성 및 사용 중인지 확인이 가능하다.

```
kubectl describe dv cirros-dv
```

```
Name:          cirros-dv
```

```
Namespace:     default
```

```
Phase:         Succeeded
```

```
Progress:      100.0%
```

```
Events:
```

```
Normal UploadScheduled ... Upload pod scheduled
```

```
Normal ImportSucceeded ... DataVolume completed successfully
```



VMI

VMI(VirtualMachineInstance)

OpenStack 비교

- VMI=실제 하이퍼바이저 위에서 돌고 있는 인스턴스 런타임
- Nova의 instance state가 Running일 때 대시보드 혹은 CLI 출력

```
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME
cirros-vm	4m	Running	10.244.0.23	kube-virt



VM 1

VM(VirtualMachine)

OpenStack 비교

- openstack server stop/start와 거의 동일한 체감
- 차이: KubeVirt는 VMI 인스턴스 런타임이라 중지 시 VMI가 없어짐

```
kubectl patch vm cirros-vm --type merge -p '{"spec":{"running":false}}'
```

```
kubectl get vm,vmi
```

NAME	AGE	STATUS
virtualmachine/cirros-vm	5m	Stopped

```
No resources found in default namespace.
```



VM 2

VM(VirtualMachine)

아래 명령어는 인스턴스를 patch를 통해서 실행 상태로 변경.

```
kubectl patch vm cirros-vm --type merge -p '{"spec":{"running":true}}'  
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME
cirros-vm	10s	Running	10.244.0.24	kube-virt



SNAPSHOT 설명

VirtualMachineSnapshot

다른 PaaS와 동일하게 Snapshot 생성 및 관리가 가능하다. 여기서는 생성부분은 다루지 않고 간단하게 확인 및 비교 한다.

OpenStack 비교

- openstack volume snapshot create 또는 openstack server image create와 비슷
- 구현은 스토리지/CSI 기능에 의존(백엔드가 지원해야 스냅샷 가능)
- 생성 및 복구는 YAML 통해서 가능
- 조회는 kubectl 명령어를 통해서 가능



SNAPSHOT 기능 활성화

기본값으로 설치한 경우, Snapshot이 동작하지 않는다. 사용하기 위해서는 Snapshot 기능을 활성화 해야 한다. 활성화를 위해서는 아래와 같이 CR에 다음과 같이 기능을 수정한다.

일반적인 운영에서는 다음과 같은 기능을 활성화 한다.

```
kubectl -n kubevirt patch kubevirt kubevirt --type='merge' -p '{
  "spec": {
    "configuration": {
      "developerConfiguration": {
        "featureGates": ["LiveMigration", "CPUManager", "Snapshot"]
      }
    }
  }
}'
```



SNAPSHOT 생성

명령어로는 스냅샷 생성은 어렵다. 생성을 위해서는 아래와 같이 YAML으로 작성 및 구성해야 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: cirros-snap
  namespace: default
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
```

EOF



SNAPSHOT 목록 확인

스냅샷이 구성이 되면 아래와 같이 확인이 가능하다.

```
kubectl get virtualmachinesnapshot
```

NAME	SOURCEKIND	SOURCENAME	PHASE	READYTOUSE	
CREATIONTIME	ERROR				
cirros-snap	VirtualMachine	cirros-vm	Succeeded	true	3m59s



SNAPSHOT 복구

명령어로는 스냅샷 생성은 어렵다. 생성을 위해서는 아래와 같이 YAML으로 작성 및 구성해야 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: cirros-snap
  namespace: default
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
```

EOF



SNAPSHOT 오픈스택과 비교

오픈스택과 기능을 비교하면 아래와 같다.

OpenStack	KubeVirt
<code>openstack server image create</code>	VirtualMachineSnapshot
<code>openstack volume snapshot create</code>	VolumeSnapshot
<code>snapshot → server restore</code>	VirtualMachineRestore
명령형 CLI	선언형 CRD



CLONE/POOL 설명

여러 개의 인스턴스를 생성 및 관리 시 사용한다. 오픈스택과 다른 부분이 많기 때문에, 사용 시 몇몇 부분은 주의 해야 한다.

1. 공유 PVC로 간단히(랩 용도)
2. DataSource+개별 VM 생성(또는 자동화 스크립트/오퍼레이터, 운영 용도)
3. Clone 여러 개+라벨 그룹 운영(운영 용도)

OpenStack 비교

- 템플릿 기반 복제=Glance 이미지로 새 서버 만들기과 유사
- KubeVirt는 VM 정의+디스크 복제가 한 덩어리로 움직임

자세한 내용은 가상머신 관리 부분에서 다시 다루도록 한다. 여기서는 간단한 예제만 다룬다.



테스트를 위한 POOL 생성

```
cat <<'EOF' | kubectl apply -f -
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
  name: cirros-clone
  namespace: default
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm-copy
EOF
```



테스트 POOL 확인 및 검증

생성이 완료가 되면, 아래 명령어로 간단하게 확인이 가능하다. 하지만, 현재 이미지는 쿠버네티스 가상화의 기능을 완전히 제공하지 않는다.

```
kubectl get vmclone
```

NAME	PHASE	SOURCEVIRTUALMACHINE	TARGETVIRTUALMACHINE
cirros-clone		cirros-vm	cirros-vm-copy

```
kubectl describe vmclone cirros-clone
```

Events:

Type	Reason	Age	From
Normal	VMVolumeSnapshotsInvalid	6m53s (x2 over 6m53s)	clone-controller

Virtual Machine volume rootdisk does not support snapshots



현재 사용중인 이미지는 rootdisk 공유가 안되는 일반 가상 머신 이미지. 이미지 빌드에서 다시 설명

골든 이미지 등록(dv)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: cirros-http-dv
  namespace: default
spec:
  pvc:
    storageClassName: nfs-client
    accessModes: ["ReadWriteMany"]
    volumeMode: Filesystem
  resources:
    requests:
      storage: 2Gi
```

자세한 설명은 운영에서 다룹니다!

계
속



골든 이미지 등록(dv)

```
source:  
  http:  
    url: https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-  
disk.img  
EOF
```



골든 기반 인스턴스 생성 1(vm)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cirros-01
  namespace: default
spec:
  running: true
  template:
    spec:
      domain:
        resources:
          requests:
            memory: 256Mi
```

간단하게 강사 설명 :)

계
속



골든 기반 인스턴스 생성 2(vm)

```
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
    volumes:
    - name: rootdisk
      persistentVolumeClaim:
        claimName: cirros-http-dv
```

EOF

```
kubectl get vm -w
```

NAME	AGE	STATUS	READY
cirros-01	1s	Starting	False
cirros-vm	6h28m	Running	True



가상머신 풀 설명(vmpool)

가상머신 웅덩이(Pool)에서는 여러 개의 가상머신을 스케일링이 가능하도록 한다. clone은 기본적으로 복제가 주요 목적이기 때문에 스케일링(scaling)이 불가능하다.

쿠버네티스의 장점인 스케일링 기능을 사용하기 위해서 VMP(VirtualMachinePool)를 통해서 이 기능을 구현해야 한다. 아래와 같이 구성 및 생성이 가능하다.

여기서는 간단하게 테스트만 한다.



가상머신 풀 간단한 예제 1(vmpool)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: pool.kubevirt.io/v1alpha1
kind: VirtualMachinePool
metadata:
  name: cirros-pool
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      kubevirt.io/vmpool: cirros-pool
```



가상머신 풀 간단한 예제 2(vmpool)

```
virtualMachineTemplate:  
  metadata:  
    labels:  
      kubevirt.io/vmpool: cirros-pool  
      app: cirros  
  spec:  
    running: true  
    template:  
      spec:  
        domain:  
          cpu:  
            cores: 1
```



가상머신 풀 간단한 예제 3(vmpool)

```
resources:
  requests:
    memory: 256Mi
devices:
  disks:
    - name: rootdisk
      disk:
        bus: virtio
volumes:
  - name: rootdisk
    persistentVolumeClaim:
      claimName: cirros-dv
```

EOF



가상머신 풀 확인 3(vmp)

아래와 같이 확인 및 스케일링이 가능하다.

```
kubectl get vmpool
```

NAME	DESIRED	CURRENT	READY	AGE
cirros-pool	2	2		48s

```
kubectl scale vmpool cirros-pool --replicas=3
```

```
virtualmachinepool.pool.kubevirt.io/cirros-pool scaled
```

```
kubectl scale vmirs cirros-pool --replicas=2
```

```
error: no objects passed to scale
```

```
virtualmachineinstancereplicaset.kubevirt.io "cirros-pool" not found
```

vmirs부분은 운영에
서 따로 다룬다.



CLONE/POOL 비교

방식	Pool 사용	디스크 분리	실습용	운영	OpenStack 대응
Pool + 공유 PVC	가능	불가능	가능	비권장	동일 이미지 NFS 부팅
Pool + 개별 PVC	비권장	불가능(자동)	비권장	비권장	직접 지원 없음
Clone 여러 개 + 라벨	불가능	가능	가능	권장	이미지로 서버 다수 생성
DataSource + VM 템플릿	불가능	가능	비권장	권장	Glance Golden Image
Harvester	가능	가능	권장	권장	Horizon 기반 운영



데이터소스(DataSource,ds) 설명

DataSource=KubeVirt식 Glance 이미지 포인터.

- 실제 데이터를 들고 있지 않음
- "이 PVC가 골든 이미지"라고 참조만 제공.
- 여러 VM이 이걸 기준으로 각자 디스크를 생성

이를 통해서 가상머신 생성이 가능하다. 또한 제공하는 소스 형식이 다양하게 제공이 된다.

1. HTTP
2. PVC
3. Docker Registry

자세한 부분은 가상머신 운영에서 다루도록 한다.



데이터소스 생성(ds)

아래와 같이 데이터소스(DataSource)에 접근이 가능하다. 먼저, 테스트 용도로 간단하게 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataSource
metadata:
  name: cirros-golden
  namespace: default
spec:
  source:
    pvc:
      name: cirros-dv
      namespace: default
```

EOF



데이터소스 조회(ds)

생성 후, 아래와 같이 조회가 가능하다.

```
kubectl get datasources
```

NAME	AGE
cirros-golden	7s



이미지 자동 갱신(DATA IMPORT CRON, dic)

이미지를 주기적으로 업데이트가 필요한 경우가 있다. 이러한 경우 DIC를 활용하여 주기적으로 Goden 이미지를 업데이트가 가능하다.

다만, DIC를 활용하기 위해서는 최소 한 개의 컨테이너 레지스트리 서버 혹은 외부에서 다운로드가 가능한 웹 서버가 필요하다.



이미지 자동 갱신 확인(dic)

사용중인 이미지를 자동으로 갱신이 되도록 하기 위해서 CRON를 구성하여 자동으로 업데이트 되게 한다.

```
kubectl get dataimportcron  
kubectl describe dic ubuntu-cron
```



이미지 자동 갱신 예제 1(dic)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataImportCron
metadata:
  name: cirros-cron
  namespace: default
spec:
  schedule: "*/30 * * * *"
  managedDataSource: cirros-latest
  template:
    spec:
```



이미지 자동 갱신 예제 2(dic)

```
source:
  registry:
    url: docker://quay.io/kubevirt/cirros-container-disk-
demo:20260201_54046b51c9-amd64
  storage:
    storageClassName: nfs-client
    accessModes:
      - ReadWriteMany
    volumeMode: Filesystem
  resources:
    requests:
      storage: 2Gi
```

EOF



이미지 자동 갱신 확인(dic)

아래 명령어로 확인이 가능하다.

```
kubectl get dataimportcron
```

```
NAME          FORMAT
```

```
cirros-cron
```

```
kubectl describe dic cirros-cron
```

```
Spec:
```

```
  Managed Data Source:  cirros-latest
```

```
  Schedule:             */30 * * * *
```

```
  Template:
```

```
    Source:
```

```
      Registry:
```

```
        URL:  docker:// quay.io/kubevirt/cirros-container-disk-  
demo:20260201_54046b51c9-amd64
```



스토리지 프로파일

가상머신이 생성 시 사용할 PVC를 선언한다.

```
kubectl get storageprofile
```

NAME	AGE
nfs-client	6h57m



이미지 업로드 토큰 설명(utr)

간단하게 모든 이미지 업로드를 위한 토큰.

UploadTokenRequest는 사용자가 디스크 이미지를 안전하게 업로드할 수 있도록 일회성 업로드 토큰(URL)을 발급하는 CDI 보안 메커니즘이다.

토큰은 사용자가 생성하는 자원은 아니며, `virtctl image-upload`와 같은 명령어를 사용 시 자동으로 생성된

kind: UploadTokenRequest



이미지 업로드 토큰 설명(utr)

왜 필요한가?

KubeVirt/CDI에서 이미지 업로드는...

1. 내부 서비스(cdi-uploadproxy)를 통해서만 가능
2. 아무나 접근하면 안 됨
3. 업로드 권한은 짧은 시간+1회성이 이상적

항목	설명
목적	이미지 업로드용 임시 인증 토큰 발급
대상	DataVolume (Upload 타입)
유효성	짧은 TTL + 1회성
보안	ServiceAccount/RBAC 기반
방법	virtctl image-upload, UI, 자동화 도구



이미지 업로드 토큰 동작 순서 설명

① UploadTokenRequest 생성

- 사용자가 “이 DV에 업로드하고 싶다” 요청
- CDI가 업로드 전용 토큰 생성

② 토큰 반환

- API 서버가 Upload URL + Token 제공
- 내부 서비스(cdi-uploadproxy) 주소 포함

③ 이미지 업로드

- 클라이언트가 토큰으로 인증
- qcow2 / raw 이미지 업로드

④ 토큰 만료

- 업로드 완료 or 시간 초과 시 자동 폐기



이미지 업로드 토큰 동작 순서

```
[User / virtctl]
|
| 1. UploadTokenRequest
v
[Kubernetes API / CDI]
|
| 2. Upload URL + Token
v
[cdi-uploadproxy Service]
|
| 3. Image Upload
v
[PVC / DataVolume]
```



가상머신 마이그레이션(vmim) 설명

VMIM은 실행 중인 VM(VMI)을 중단 없이 다른 노드로 이동시키는 라이브 마이그레이션 리소스다. 이 기능은 다른 솔루션도 사용하는 마이그레이션과 동일하다.

VMIM은 다음과 같은 기능을 제공한다.

1. 실행 중인 VirtualMachineInstance(VMI) 다른 노드로 이동
2. 이동 시 다운타임 최소화
3. Kubernetes 컴퓨트 노드로 이동(클러스터 이동은 미지원)



가상머신 마이그레이션(vmim) 활용

마이그레이션 기능 활용 방법은 두 가지가 있다.

1. 명령어로 실행
2. YAML으로 선언 및 실행

```
kubectl get vmim
```

NAME	PHASE	VMI
cirros-migration	Succeeded	cirros-vm

Multus나 혹은 Bridge네트워크로 변경해야 한다.

```
virtctl migrate cirros-vm
```

Message: cannot migrate VMI which does not use masquerade, bridge with kubevirt.io/allow-pod-bridge-network-live-migration VM annotation or a migratable plugin to connect to the pod network



가상머신 마이그레이션 네트워크

쿠버네티스 가상화에서는 네트워크를 다음과 같이 구성을 권장한다.

네트워크 방식	브릿지 직접 생성	Migration
Pod + bridge	아니요	기본적으로 불가능
Pod + masquerade	아니요	가능
Multus + Linux bridge	가능	구성에 따라 불가능
Multus + SR-IOV	PF/VF 필요	미지원



가상머신 복제(vmirs) 설명 1

VMIRS는 동일한 스펙의 VMI를 지정한 개수만큼 유지하는 VMI 전용 ReplicaSet이다. 그러면, VMIRS가 어떠한 기능을 구현하고 기존 쿠버네티스 기능과 무엇이 비슷한가?

1. VMI(VirtualMachineInstance) 를 대상으로 함
2. 지정한 replicas 수를 항상 유지
3. 하나가 죽으면 새 VMI를 자동 생성
4. 기본적으로 쿠버네티스의 ReplicaSet과 동일



가상머신 복제(vmirs) 설명 2

직접적으로 가상머신을 생성하지 않고, `replicas:`의 개수를 유지하려고 한다. 그리고, 이 기능의 치명적(?)인 단점은 다음과 같다.

1. 디스크 문제

- VMIRS는 공유 디스크 전제
- 대부분 RWX PVC 필요
- VM별 디스크 분리 불가능

VMIRS (디스크 공유)

```
├─ VMI #1 (Running)
├─ VMI #2 (Running)
└─ VMI #3 (Running)
```



가상머신 복제(vmirs) 설명 3

2. 아래와 같은 관리 기능은 사용이 불가능

- Snapshot
- Clone
- Start/Stop

3 .Live Migration과 궁합이 나쁨

- Pod 네트워크 제약이 많음(마이그레이션은 불가능)
- 장애 시 마이그레이션이 아니라 재생성



가상머신 복제(vmirs) 예제 1

```
cat <<EOF> | kubectl apply -  
apiVersion: kubevirt.io/v1  
kind: VirtualMachineInstanceReplicaSet  
metadata:  
  name: cirros-vmirs  
  namespace: default  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: cirros
```



가상머신 복제(vmirs) 예제 2

```
template:  
  metadata:  
    labels:  
      app: cirros  
  spec:  
    instancetype:  
      name: medium
```



가상머신 복제(vmirs) 예제 3

```
domain:
  devices:
    disks:
      - name: rootdisk
        disk:
          bus: virtio
    interfaces:
      - name: default
        masquerade: {}
```



가상머신 복제(vmirs) 예제 4

```
networks:  
  - name: default  
    pod: {}  
volumes:  
  - name: rootdisk  
    persistentVolumeClaim:  
      claimName: cirros-rwx-pvc
```

EOF



가상머신 복제(vmirs) 예상 결과

```
kubectl get vmirs
```

NAME	DESIRED	CURRENT	READY	AGE
cirros-vmirs	3	3	3	30s

```
kubectl get vmi -l app=cirros -o wide
```

NAME	PHASE	IP	NODE
cirros-vmirs-xxxxx	Running	10.244.0.51	node-1
cirros-vmirs-yyyyy	Running	10.244.1.12	node-2
cirros-vmirs-zzzzz	Running	10.244.2.33	node-3



가상머신 내보내기(vmexport) 설명

VirtualMachineExport (vmexport), KubeVirt에서 VM(또는 PVC)의 디스크를 다운로드 가능한 형태로 내보내는(Export) 기능.

이 기능은 VM 백업/다른 환경으로 이동 시 사용한다. 백업 시, 두 가지 방법으로 백업이 가능하다.

1. VM (VM에 연결된 볼륨을 export)
2. PVC (특정 디스크/PVC를 직접 export)

이미지 파일로 다운로드 받기 위해서는 생성 후 제공해주는 URL로 다운로드가 가능하다. 가상머신 운영 부분에서 자세히 다루도록 한다.



가상머신 사양 설명(vmf)

VirtualMachineInstanceType은 VM의 하드웨어 스펙(Flavor)을 정의한다. 오픈스택에서 Flavor 기능과 동일하다. 선언이 안되어 있는 경우 가상머신 생성 시 명시해야 한다.

1. 인스턴스 사양 선언
2. 직접적으로 VM/VMI에 사양 선언

실 서비스 구성 시, 가급적이면 vmf 기반으로 구성한다.



가상머신 사양 직접 명시 예제

아래는 앞에서 구성 시 사용하였던 직접 하드웨어 사양 명시.

```
spec:
  running: true
  template:
    spec:
      domain:
        resources:
          requests:
            cpu: "1"
            memory: 1Gi
```



인스턴스 타입 선언(vmf)

아래와 같이 인스턴스 타입을 선언 및 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion:instancetype.kubevirt.io/v1beta1
kind: VirtualMachineInstancetype
metadata:
  name: medium
  namespace: default
spec:
  cpu:
    guest: 2
  memory:
    guest: 4Gi
EOF
```



가상머신 사양 선언(vmf)

선언이 되어 있는 경우, 아래와 같이 하드웨어 사양 선언.

```
kubectl get vmf
```

```
NAME      AGE
```

```
medium    52s
```

```
spec:
```

```
  instancetype:
```

```
    name: small
```



가상머신 권장 설정 설명(VMP)

VMP는 VMF와 비슷하지만, 차이점은 사용하는 가상머신 이미지에 따라서 자동으로 가상머신 사양을 선택한다. 오픈스택과 비교하면, Flavor에 메타정보를 오버라이드 하는 기능과 동일하다.

예를 들어 다음과 같이 VMF를 설정한다.

- 디스크 버스 (virtio/sata)
- NIC 모델
- 머신 타입 (q35 등)
- OS 타입 힌트 (linux/windows)

이 기능은 권장사항, 강제적으로 적용되는 자원은 아니다. 사용자에게 좀 더 나은 하드웨어 선택 환경을 제공한다.

- `preferredDiskBus: virtio`
- `preferredInterfaceModel: virtio`



가상머신 권장 설정 선언(VMP)

아래와 같이 가상머신 사양 덮어쓰기가 가능하다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: instancetype.kubevirt.io/v1beta1  
kind: VirtualMachinePreference  
metadata:  
  name: linux  
  namespace: default
```



가상머신 권장 설정 선언(VMP)

아래와 같이 가상머신 사양 덮어쓰기가 가능하다.

```
spec:
  devices:
    preferredDiskBus: virtio
    preferredInterfaceModel: virtio
  machine:
    preferredMachineType: q35
  firmware:
    preferredUseEfi: false
```

EOF



가상머신 권장 설정 선언 확인(VMP)

아래와 같이 확인이 가능하다. 사용 방법은 가상머신 운영 부분에서 다시 다루도록 한다.

```
kubectl get vmp
```

```
NAME    AGE
```

```
linux   46s
```



VMF/VMP 비교

구분	InstanceType (vmf)	Preference (vmp)
역할	스펙 정의	권장 설정
성격	강제	힌트
CPU / Memory	포함	없음
디바이스 설정	불가능	가능
오버라이드	불가능	가능
선택 가능	강제	선택사항
OpenStack 대응	Flavor	Image metadata



오픈스택과 자원 비교

KubeVirt/CDI	의미	OpenStack 대응
DataVolume(dv)	이미지 → PVC로 가져오기	Glance 이미지로 Cinder 볼륨 생성(유사)
PVC	VM 디스크	Cinder Volume
VM(virtualmachine)	VM 정의 + 전원	Nova Server Status
VMI(virtualmachineinstance)	실행 인스턴스 런타임	Nova instance
vmsnapshot/vmrestore	스냅샷/복원	volume snapshot/rollback 또는 server image
vmclone	VM 복제	이미지 기반 새 서버 생성(유사)
vmpool/vmirs	VM 스케일아웃	Heat AutoScaling(유사)
StorageClass/StorageProfile	스토리지 정책	Cinder backend/type



마지막 virtctl 명령어

virtctl 명령어는 가상머신 구성 시 자원 설정 및 수정을 위해서 사용하는 명령어. 이 두 가지 명령어는 다음과 같은 차이가 있다. 이 명령어에 대해서는 교육을 진행하면서 사용한다.

1. kubect, 리소스 관리 및 조작
2. virtctl, VM 조작(운영에 더 가까움)

구분	kubectl	virtctl
역할	리소스 관리	VM 조작
대상	CR, YAML	실행 중 VM
콘솔	미지원	가능
전원 제어	미지원	가능
마이그레이션	미지원	가능
교육 난이도	높음	낮음



디스크 이미지

이미지 빌드



설명

디스크 이미지



쿠버네티스 가상화에서 디스크 이미지 개념의 변화

전통적인 가상화 플랫폼인 OpenStack, VMware, Red Hat Virtualization(RHV)에서는 디스크 이미지는 서로 각기 다른 형태로 운영이 되었다. 이러한 이유로 서로 호환이 안되는 부분이 있었고, 이를 해결하기 위해서 이미지 변환이 필요했다.

하지만, 쿠버네티스 가상화에서는 모든 이미지는 두 가지 형태로 사용이 가능하다.

1. 컨테이너 이미지(OCI/Docker Image)
2. QCOW2 이미지(Libvirt, QEMU Image)

운영자는 사전에 qcow2 또는 raw 형태의 이미지를 플랫폼 내부 저장소에 등록하고, 가상머신 생성 시 해당 이미지를 복제(clone) 하여 디스크를 만든다.

이 구조에서는 이미지 안에 OS 설정, 사용자 계정, 네트워크 초기 설정까지 상당 부분이 미리 포함이 되어 있거나, 혹은, Cloud-Init를 사용하여 초기화가 가능하다.



쿠버네티스 가상화(Kube-Virt)의 디스크 이미지 방식

쿠버네티스 가상화 환경(Kube-Virt)에서는 이 관점이 완전히 달라진다.

디스크 이미지는 더 이상 하이퍼바이저가 제공하는 자원이 아니라, 쿠버네티스 스토리지 리소스(PVC)를 초기화하기 위한 재료에 가깝다.

가상머신은 이미지를 복제해서 생성되지 않는다. 이미지는 한 번 PVC로 변환되고, 가상머신은 해당 PVC를 볼륨으로 연결 및 구성이 되어 실행된다.

이 구조에서는 다음과 같은 조건으로 동작한다.

- 디스크의 실제 크기와 수명은 PVC가 결정하고
- VM은 디스크의 소유자가 아니라 소비자에 가깝다

따라서 이미지에는 최소한의 OS만 포함하고, 사용자 계정, SSH 키, 네트워크 설정, 초기 스크립트 등은 VM Spec과 cloud-init을 통해 선언적으로 주입하는 방식이 권장된다



지원 디스크 이미지 형식 설명 1

현재 쿠버네티스는 다음과 같은 이미지 형식을 기본값으로 제공하고 있다.

1. QCOW2/3
2. RAW
3. OCI Container Image

가상머신 이미지는 Libvirt처럼 바로 사용이 불가능하다. OCI Image로 한번 더 패키징 후 사용이 가능하다. 이러한 이유로 아래와 같은 조건이 한번 더 적용이 된다.

- 컨테이너 이미지 안에 raw/qcow2 디스크 파일을 포함
- 해당 컨테이너 이미지를 DataVolume 소스로 사용
- VM 생성 시 컨테이너 레지스트리에서 이미지를 pull하여 디스크를 생성
- 이미지 관리를 위한 컨테이너 레지스트리 서버



지원 디스크 이미지 형식 설명 2

이와 같은 방식으로 구성이 되는 경우, 기존 쿠버네티스 환경을 그대로 사용이 가능하기 때문에 다음과 같은 장점을 가지고 있다.

- 레지스트리 기반 배포
- CI/CD 파이프라인과의 자연스러운 연계
- 이미지 버전 관리의 용이성



지원 디스크 이미지 형식 설명 3

하지만 운영 관점에서는 다음과 같은 한계가 있다.

- 컨테이너 이미지 레이어 구조는 대용량 디스크에 비효율적
- 스토리지 성능 및 관리 가시성이 떨어질 수 있음
- VM 디스크와 컨테이너 이미지의 수명 주기가 어긋나기 쉬움

이전에 사용하던 가상머신 기반의 PaaS 플랫폼과 다르게, 이미지 관리가 좀 더 복잡해지는 단점이 발생한다.



이미지 CI/CD 설명

쿠버네티스와 동일하게 CI/CD 기능을 가상머신 이미지에 적용이 가능하다. 다만, 가상머신 이미지 빌드라서 컨테이너 이미지처럼 빠른 생성 및 배포는 어려운 부분이 있다.

가상머신을 구성하기 위해서 다음과 같은 내용들이 반드시 가상머신에 포함이 되어야 한다.

- VirtioIO 드라이버(net, blk)
- cloud-init, ignition

또한, 이미지 빌드를 CI에 포함시키기 위해서 다음과 같은 도구를 권장한다.

- 디스크 이미지 빌드(Disk Image Builder)
- Packer



이미지 CI/CD 설명 2

모든 리눅스 배포판은 보통 Packer는 라이선스 문제로 포함이 안되어 있다. 이러한 이유로 둘 중 하나의 디스크 이미지 빌드 도구 사용을 권장한다.

1. `dib(QCOW2)`
2. `oz(Legacy)`
3. `buildah(Container Format)`



기존 가상화와 쿠버네티스 가상화 이미지 비교

항목	오픈스택	VMWARE	쿠버네티스 가상화
이미지 소유	Glance	하이퍼바이저	Kubernetes Container Registry
디스크 생성	Cinder	하이퍼바이저	PVC attach
설정 주체	Flavor	하이퍼바이저	VMF
네트워크	Neutron	하이퍼바이저	CNI
확장	Nova-compute	VM 단위	스토리지 리소스 단위
관리 관점	인프라	인프라	선언형 리소스



디스크 빌드

디스크 이미지



디스크 이미지 빌드

이 과정에서는 간단하게 DIB(Disk Image Builder)를 통해서 생성 및 구성한다. 본래 이 도구는 Libvirt나 혹은 OpenStack에 많이 사용한 도구이다.

사용방법이 생각보다 간단하며, 기본적인 elements 환경을 제공하며, 사용자가 원하는 내용은 추가적으로 구성하여 적용이 가능하다.



핵심 구성 요소 (Core Components)

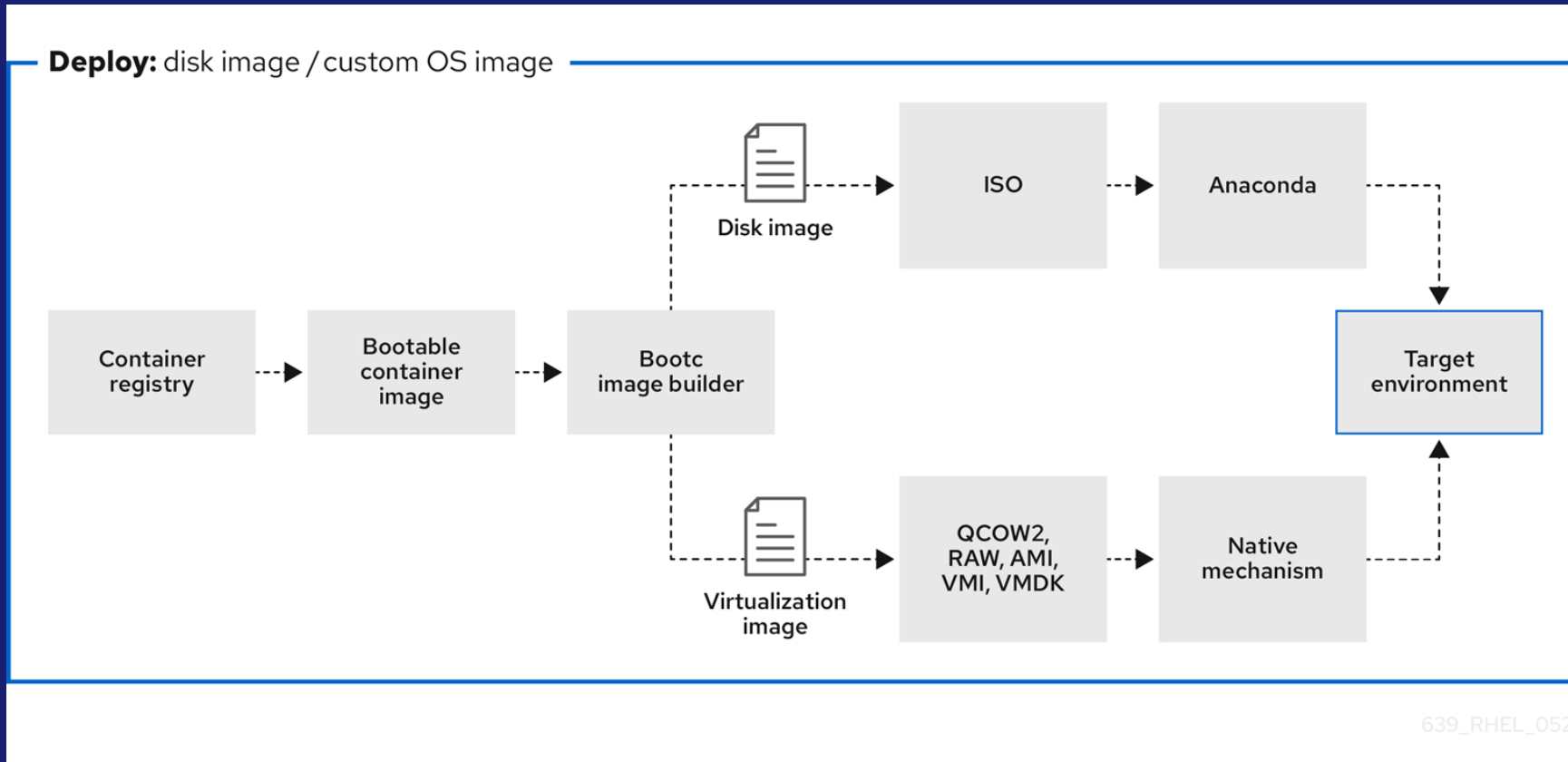
DIB의 가장 핵심적인 구성요소 단위는 다음과 같다. 요소(Elements)는 특정 기능을 수행하는 스크립트와 설정 파일의 모음.

1. **Base Elements:** ubuntu, fedora, centos와 같은 OS의 기본 이미지를 정의
2. **Feature Elements:** cloud-init, dhcp-all-interfaces, bootloader와 같은 기본 기능 정의
3. **Service Elements:** apache, mysql, tomcat과 같은 서비스 소프트웨어 설치 및 정의

디스크 이미지 빌드는 이미지 빌드 시, 컨테이너에서 많이 사용하는 베이스 이미지 기반으로 설치를 진행한다. 기본 이미지로 설치 후 최종적으로 부트로더를 가상머신 이미지에 같이 포함한다.



디스크 이미지 빌드 및 배포



639_RHEL_0524



기본적인 구성요소

분류	구성 요소	설명
핵심 단위	Elements	OS, 패키지 설치, 설정 등 특정 기능을 담은 스크립트 모음
실행 도구	disk-image-create	실제 디스크 이미지를 생성하는 메인 CLI 도구
실행 도구	ramdisk-image-create	배포용 램디스크(예: Ironiic IPA) 생성 전용 도구
빌드 단계	Phases	root.d부터 cleanup.d까지 스크립트가 실행되는 순서
제어 변수	Environment Variables	OS 버전, 아키텍처, 커널 등 빌드 옵션을 결정하는 환경 변수
결과 형식	Image Formats	qcow2, raw, vmdk 등 최종 이미지 파일의 포맷
빌드 메커니즘	Chroot	격리된 환경에서 이미지를 구성하기 위한 가상 루트 환경



빌드 단계

단계 (Phase)	설명
root.d	파일시스템의 기본 구조를 만들고 패키지를 설치할 준비를 합니다.
extra-data.d	호스트 환경의 파일을 이미지 내부로 복사합니다.
pre-install.d	패키지 설치 전 필요한 설정을 수행합니다.
install.d	실제 패키지를 설치하거나 소스 코드를 빌드합니다.
post-install.d	설치 후 마무리 설정 및 최적화를 진행합니다.
finalise.d	이미지 생성 마지막 단계로, 정리 작업을 수행합니다.
cleanup.d	빌드 과정에서 발생한 임시 파일을 삭제합니다.



설치 및 확인

다음과 같이 패키지를 설치한다. 설치가 완료가 되면 elements 디렉터로 같이 확인한다. 지원하는 배포판 목록은 아래처럼 확인이 가능하다.

```
dnf install -y diskimage-builder
```

```
ls -sl /usr/share/diskimage-builder/elements/
```

```
ls /usr/share/diskimage-builder/elements/ | grep -e rocky -e rhel -e centos  
-e ubuntu
```



DIB에서 지원하는 배포판

지원 배포판 목록은 다음과 같다.

배포판	element 이름	지원여부
Ubuntu	ubuntu	지원
Debian	debian	지원
CentOS	centos	지원
Rocky Linux	rocky	지원
AlmaLinux	almalinux	미지원
RHEL	rhel	지원
Fedora	fedora	지원
openSUSE	opensuse	미지원
SLES	sles	미지원
Cirros	cirros	미지원



간단하게 이미지 빌드 설명

기존 방법처럼 virt-builder, virt-install, virt-sparsify를 사용하여도 문제는 없다. 다만, 골덴(Golden)이미지를 만들기 위해서는 가급적이면 스크래치(Scratch)로 구성을 권장한다.

사용하는 방법은 다음과 같다.

```
disk-image-create <base-element> [추가-element ... ] -o <출력이미지명>  
disk-image-create cirros -o cirros
```

보통 옵션에 많이 사용하는 옵션은 -o, -t.

- -o: output
- -t: disk format/type



간단하게 이미지 빌드 1

빌드 시, 다음과 같이 이미지 빌드 실행을 한다.

```
export DIB_RELEASE=9  
disk-image-create rocky-container vm -t qcow2 -o rocky9
```

위의 내용을 실행 하였을 때, elements가 선언이 안되어 있어서 올바르게 이미지 빌드가 불가능 하다. 올바르게 사용하기 위해서 아래와 같이 추가적으로 선언이 필요하다. 빌드 시, 컨테이너 이미지를 기반으로 진행하기 때문에 다음과 같은 도구를 추가적으로 설치해야 한다.

```
dnf install -y podman buildah  
export ELEMENTS_PATH=$HOME/elements:/usr/share/diskimage-builder/elements  
disk-image-create rocky-container vm -t qcow2 -o rocky9
```



간단하게 이미지 빌드 2

위와 같이 진행하는 경우, 기본 옵션이 적용이 되면서 실행이 된다. 만약, 사용자가 원하는 기능만 적용하고 싶은 경우 아래와 같이 적용이 가능하다.

```
export DIB_RELEASE=9
export ELEMENTS_PATH=$HOME/elements:/usr/share/diskimage-builder/elements
disk-image-create rocky-container \
    devuser \
    vm \
    cloud-init \
    -t qcow2 \
    -o rocky9-custom
```



간단하게 이미지 빌드 2 옵션

위의 옵션의 기능은 다음과 같다.

Element 이름	역할	설명
rocky	OS 베이스	Rocky Linux 기본 OS 설치
devuser	사용자 커스터마이징	사용자 계정 생성, 비밀번호 설정 등 사용자 추가 작업(뒤에 표 참고)
vm	VM 최적화	가상머신 환경에 맞는 기본 설정(qemu-guest-agent 등)
cloud-init	초기 설정 지원	부팅 시 cloud-init을 통해 SSH 키, 네트워크, 사용자 설정 가능



간단하게 이미지 빌드 2 옵션

devuser는 기본적으로 아래와 같이 제공이 된다. 컨테이너 이미지 기반으로 빌드가 되기 때문에 아래와 같이 구성 및 생성이 되어야 한다. 다만, 기본적으로 비밀번호 로그인은 허용이 되지 않는다.

배포판	관행적 기본 계정	배포판
Ubuntu	ubuntu	Ubuntu
Debian	debian	Debian
CentOS/Rocky/Alma	cloud-user	CentOS/Rocky/Alma
Fedora	fedora	Fedora
Cirros	cirros/cubswin:)	Cirros



간단하게 이미지 빌드 3

아래 빌드를 그대로 사용하고, 테스트 시 접근할 사용자 계정 및 비밀번호를 통해서 접근이 가능하도록 한다.

```
export DIB_RELEASE=9
disk-image-create rocky-container \
  devuser \
  vm \
  cloud-init \
  -t qcow2 \
  -o rocky9-custom
```



클라우드 계정 생성 1

아래와 같이 빌드 시, 이미지에 사용자 계정에 비밀번호를 설정한다.

```
mkdir -p $HOME/elements/devuser/install.d  
cat <<'EOF' > $HOME/elements/devuser/install.d/10-create-user  
#!/bin/bash  
set -eux
```

```
useradd -m devuser  
echo devuser:devuser | chpasswd
```



클라우드 계정 생성 2

```
# sudo 권한 (선택)
```

```
echo "devuser ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/devuser
```

```
chmod 440 /etc/sudoers.d/devuser
```

```
EOF
```

```
chmod +x $HOME/elements/devuser/install.d/10-create-user
```

```
export ELEMENTS_PATH=$HOME/elements:/usr/share/diskimage-builder/elements
```

```
disk-image-create rocky devuser -o rocky-devuser
```



RL9에는 다음과 같은 버그가 있음...

이미지 빌드 시, rocky9에서 curl 문제가 발생한다. 이를 해결하기 위해서 아래와 같이 수정한다.

```
mkdir -p $HOME/elements/rocky-curl-fix/install.d
cat <<'EOF' > $HOME/elements/rocky-curl-fix/install.d/10-create-user
#!/bin/bash
set -eux
if grep -qE 'Rocky Linux release 9|AlmaLinux release 9|Red Hat Enterprise
Linux release 9' /etc/redhat-release 2>/dev/null; then
    # 혹시 dnf가 없거나 swap 실패해도 빌드 전체가 바로 죽지 않도록 || true
    dnf swap -y curl-minimal curl --allowerasing || true
fi
EOF
```



다시 이미지 빌드

아래 명령어로 다시 이미지 빌드를 시작한다. 이와 같이 시작하면 문제 없이 가상머신 이미지가 생성이 된다.

```
export DIB_RELEASE=9
disk-image-create rocky-container \
  devuser \
  vm rocky-curl-fix \
  cloud-init \
  -t qcow2 \
  -o rocky9-custom
```



확인 과정

이미지가 올바르게 구성이 되었는지 guestfish-tools-c를 통해서 확인한다.

```
dnf install -y guestfs-tools libguestfs-bash-completion libvirt
source /etc/profile.d/guestfish.sh
systemctl enable --now libvirtd
export LIBGUESTFS_BACKEND=direct
guestfish --ro -a rocky9.qcow2
><fs> run
><fs> list-partitions
><fs> mount /dev/sda1
><fs> ls /
```



이미지 패키징

이미지가 올바르게 빌드가 되었으면, OCI이미지로 패키징 한다. 패키징을 위해서 다음과 같은 도구가 필요하다.

1. Containefile 혹은 Dockerfile 빌드를 위한 런타임(Podman, Docker)
2. Buildah 설치가 가능한 배포판
3. 쿠버네티스에서 제공하는 디스크 빌드 이미지(옵션)

위와 같은 도구가 설치가 되어 있으면 바로 사용이 가능하다.

```
dnf install -y buildah
dnf install -y container-tools
```



가상머신 이미지 생성

아래 명령어로 QOCW2를 OCI로 패키징한다.

```
mkdir -p rocky9-containerdisk
cp rocky9.qcow2 rocky9-containerdisk/
cd rocky9-containerdisk
cat <<EOF> Containerfile
FROM quay.io/kubevirt/container-disk-v1alpha
COPY rocky9.qcow2 /disk/rocky9.qcow2
RUN chmod 0444 /disk/rocky9.qcow2
EOF
podman build -t rocky9-containerdisk:1.0 -f Containerfile .
```



가상머신 이미지 업로드

레지스트리 서버에 업로드 한다. 내부에 registry-v2 혹은 Quay.io에 업로드 한다.

```
mkdir -p /opt/registry/data
podman run -d --name registry --restart=always \
-p 5000:5000 -v /opt/registry/data:/var/lib/registry:z registry:latest

podman tag rocky9-containerdisk:1.0 10.0.4.131:5000/kubevirt/rocky9-
containerdisk:1.0
podman push --tls-verify=false 10.0.4.131:5000/kubevirt/rocky9-
containerdisk:1.0
```



INSECURE TLS

INSECURE TLS 구성에서는 아래와 같이 구성한다.

```
mkdir -p /opt/registry/data
podman run -d --name registry --restart=always \
-p 5000:5000 -v /opt/registry/data:/var/lib/registry:z registry:latest

podman tag rocky9-containerdisk:1.0 10.0.4.131:5000/kubevirt/rocky9-
containerdisk:1.0
podman push --tls-verify=false 10.0.4.131:5000/kubevirt/rocky9-
containerdisk:1.0
```



가상머신 이미지 적용 1

이미지가 올바르게 생성 및 업로드가 되었으면 아래 명령어로 확인한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: rocky9-cdisk-vm
  namespace: default
spec:
  running: true
  template:
    spec:
```



가상머신 이미지 적용 2

```
domain:
  resources:
    requests:
      memory: 2Gi
  devices:
    disks:
      - name: rootdisk
        disk:
          bus: virtio
  interfaces:
    - name: default
      masquerade: {}
```



가상머신 이미지 적용 3

```
networks:  
- name: default  
  pod: {}  
volumes:  
- name: rootdisk  
  containerDisk:  
    image: 10.0.4.131:5000/kubevirt/rocky9-containerdisk:1.0
```

EOF

```
kubectl get vm,vmi
```



스토리지 및 네트워크

네트워크 설정



쿠버네티스 가상화 네트워크

쿠버네티스 가상화 네트워크는 기본적으로 3가지 유형으로 제공한다.

1. KubeVirt Masquerade
2. KubeVirt Bridge(Pod)
3. Pod Network

쿠버네티스에서 별도의 네트워크를 구성하지 않으면 기본적으로 Pod 네트워크 기반으로 동작한다. 앞서 이야기 하였지만, KubeVirt에서 Pod Network를 사용하는 경우, 마이그레이션 기능을 활용 할 수가 없다.



가상머신 네트워크 확인

현재 가상머신이 어떠한 네트워크를 사용하는지 확인하기 위해서 아래와 같이 명령어를 실행한다.

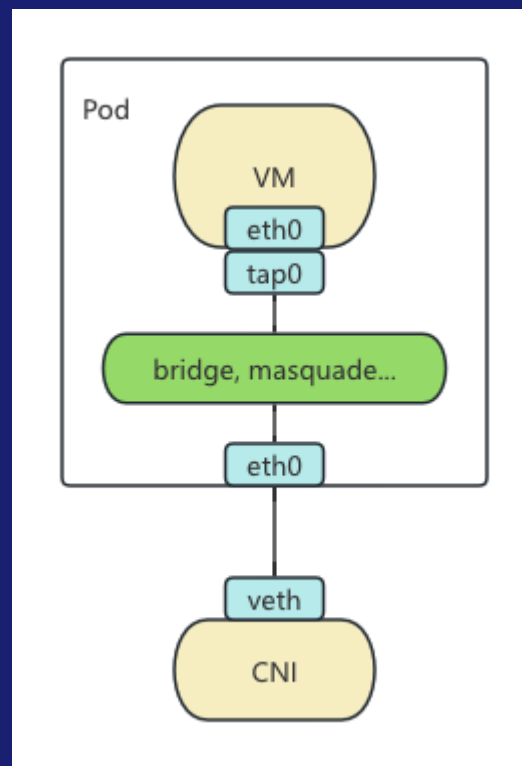
```
kubectl get vmi cirros-vm -o yaml | grep -E 'masquerade|bridge|multus'  
- bridge: {}
```

```
message: cannot migrate VMI which does not use masquerade, bridge with  
kubevirt.io/allow-pod-bridge-network-live-migration
```

```
message: 'InterfaceNotLiveMigratable: cannot migrate VMI which does not  
use masquerade, bridge with kubevirt.io/allow-pod-bridge-network-live-  
migration VM annotation'
```



마스커레이딩 네트워크 구성



네트워크 기능 확장

이러한 이유로 쿠버네티스 가상화 구성 및 확인이 완료가 되면, 네트워크 기능 확장을 시작한다. 앞에서 이야기 하였지만, 최소 권장 구성요소로 Masquerade 기반으로 구성한다.

```
kubectl get vmi cirros-vm -o yaml | grep -E 'masquerade|bridge|multus'  
- bridge: {}
```

```
message: cannot migrate VMI which does not use masquerade, bridge with  
kubevirt.io/allow-pod-bridge-network-live-migration
```

```
message: 'InterfaceNotLiveMigratable: cannot migrate VMI which does not  
use masquerade, bridge with kubevirt.io/allow-pod-bridge-network-live-  
migration VM annotation
```



가상머신에 네트워크 옵션 변경

마스커레이딩 네트워크 구성 및 선언은 아래와 같이 선언한다. 이 내용 기반으로 VM에 적용하면 다음과 같다.

```
devices:
  interfaces:
    - name: default          # ← 인터페이스 이름
      masquerade: {}
networks:
- name: default             # ← 같은 이름
  pod: {}
```



진행 전 가상머신 제거

아래 명령어로 가상머신 제거.

```
kubectl delete vm --all  
virtualmachine.kubevirt.io "cirros-01" deleted  
virtualmachine.kubevirt.io "cirros-vm" deleted
```



가상머신 생성(마스커레이드 기반)

아래 명령어로 가상머신을 마스커레이드 기반으로 구성 및 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cirros-masq
  namespace: default
spec:
  running: true
  template:
    metadata:
      labels:
        kubevirt.io/domain: cirros-masq
```



가상머신 생성(마스커레이드)

```
spec:
  domain:
    resources:
      requests:
        memory: 256Mi
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
```



가상머신 생성(마스커레이드)

```
    interfaces:
      - name: default
        masquerade: {}
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      containerDisk:
        image: quay.io/kubevirt/cirros-container-disk-demo
```

EOF



가상머신 네트워크 정보 확인

올바르게 네트워크가 구성 되었는지 아래 명령어로 확인한다.

```
kubectl get vmi
```

```
kubectl get nodes \
-o jsonpath='{range .items[*]}{.metadata.name}{ "\t" }{.spec.podCIDR}{ "\n" }'
```

클러스터 POD 아이피 확인

```
virtctl console cirros-masq
```

가상머신 콘솔 접근

```
kubectl get vmi cirros-masq \
-o jsonpath='{.spec.domain.devices.interfaces[?(@.name=="default")]}';
echo {"masquerade":{}},"name":"default"}
```

가상머신 네트워크 구성 확인



예상되는 네트워크 구성 정보

올바르게 네트워크가 구성 되었는지 아래 명령어로 확인한다.

```
- masquerade: {}  
  networks:  
    interfaces:  
      - infoSource: domain  
        ipAddress: 172.16.236.162  
        ipAddresses:  
          - 172.16.236.162  
        linkState: up  
        mac: ca:82:4c:44:2e:01  
        name: default  
        podInterfaceName: eth0  
        queueCount: 1
```

가상머신 아이피

가상머신에 연결된 인터페이스 이름



클러스터 및 마스커레이드 네트워크 확인

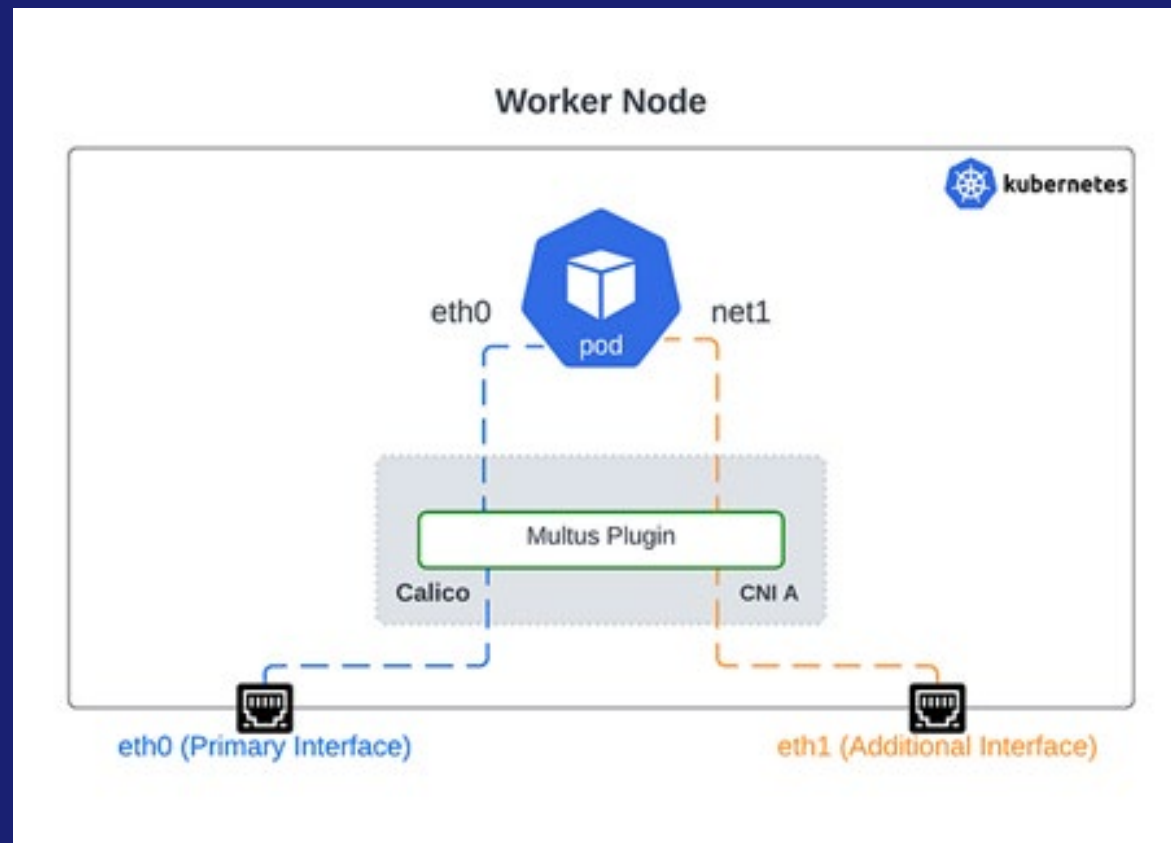
클러스터 아이피를 올바르게 받았는지 확인한다.

```
kubectl get nodes \
-o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.podCIDR}{"\n"}'
kubevirt.example.com      172.16.0.0/24
```

```
virtctl console cirros-masq
> hostname -i
> 10.0.4.131
```



Multus 설명



Multus CNI 개요

1. Multus CNI는 Kubernetes Pod에 여러 개의 네트워크 인터페이스(NIC) 를 붙이기 위한 Meta CNI 플러그인
2. 기본 CNI 위에서 동작하며, Pod 하나 = 네트워크 하나라는 기존 Kubernetes 네트워크 모델을 확장



왜 Multus가 필요했을까?

Kubernetes 기본 CNI (Flannel, Calico 등)

- Pod당 단일 네트워크 인터페이스
- 대부분 Overlay 네트워크

하지만 현실 세계의 요구는...

- 스토리지 전용 네트워크
- 고성능 데이터 플레인(SR-IOV, DPDK)
- 관리/제어망 분리
- VM/KubeVirt, NFV, Telco 워크로드

이러한 요구사항을 하나로 묶어서 처리하는게 주요 목적



동작방식

기본적인 CNI에 추가 CNI를 조합하여 네트워크를 구성한다.

1. Pod 생성 요청
2. Multus가 기본 CNI 호출
→ eth0 (Pod 기본 네트워크)
3. Pod Annotation 확인
4. 추가 CNI(macvlan, sriov, bridge 등) 호출
→ net1, net2 ... 생성
5. Pod 내부에 다중 NIC 구성 완료
6. 핵심 구성 요소



설치 전 확인 사항

Multus는 단독 CNI가 아님. 반드시 기본 CNI가 먼저 설치되어 있어야 함. 반드시, 기존에 사용하던 쿠버네티스 클러스터가 올바르게 동작이 되어야 됨.

기본 CNI 중 하나 설치됨. 이 과정에서는 Calico기반으로 구성이 되어 있음.

- Flannel
- Calico
- Cilium 등

/etc/cni/net.d/ 디렉터리에 설정이 존재해야 됨.



Multus 설치 및 확인

설치 방법은 매우 간단하다. 오픈소스에서는 설치 기능은 Manifest기반으로 권장하고 있다.

```
kubectl apply -f \
https://raw.githubusercontent.com/k8snetworkplumbingwg/multus-
cni/master/deployments/multus-daemonset.yml
```

```
kubectl get pod -n kube-system -w
kube-multus-ds-tcmhh          1/1      Running    0              32s
ls /etc/cni/net.d/
00-multus.conf
10-calico.conflict
10-flannel.conflist
```



Multus 확인

올바르게 CRD가 구성이 되었는지 아래처럼 확인한다. 여기까지 확인이 되면 설치는 완료가 되었다.

```
kubectl get crd | grep network-attachment
```



Multus 지원 영역

네트워크 유형	사용 CNI	특징	주요 사용 사례
Overlay 네트워크	Flannel, Calico, Cilium	Pod 간 가상 네트워크	기본 Pod 통신
macvlan	macvlan CNI	물리 NIC에 직접 연결	외부망 직접 연결
ipvlan	ipvlan CNI	macvlan보다 단순	대규모 Pod IP 구성
bridge	bridge CNI	Linux bridge 사용	KubeVirt VM 브릿지
SR-IOV	sriov CNI	NIC 가상화, 고성능	Telco, NFV, DPDK
Host-device	host-device CNI	물리 NIC 직접 할당	특수 HW 워크로드
OVN / OVS	ovn-kubernetes	SDN 기반 L2/L3	OpenStack 연계
DPDK	dpdk CNI	Kernel bypass	초저지연 처리
PTP (Passthrough)	SR-IOV PTP	NIC 직접 패스스루	실시간 트래픽
Bond/VLAN	vlan, bond CNI	L2 구성 확장	엔터프라이즈 네트워크



오픈스택에서 구성

오픈스택에서 Multus에서 구성하는 vlan주소를 macvlan으로 올바르게 구성이 불가능하다. 이러한 이유로 아래 처럼 네트워크를 별도로 생성 후 사용한다.

[OpenStack Neutron]

net-infra	→ 기본 Pod 네트워크
net-multus	→ Multus(macvlan)용



macvlan 네트워크 구성

아래와 같이 네트워크를 구성한다. 인터페이스 이름이 다른 경우, 인터페이스 이름을 변경한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: ext-macvlan-192-168-20
  namespace: default
spec:
  config: |
    {
      "cniVersion": "0.3.1",
      "type": "macvlan",
      "master": "ens7",
```



macvlan 네트워크 구성

```
"mode": "bridge",  
"ipam": {  
  "type": "host-local",  
  "subnet": "192.168.20.0/24",  
  "rangeStart": "192.168.20.50",  
  "rangeEnd": "192.168.20.200",  
  "gateway": "192.168.20.1",  
  "routes": [  
    { "dst": "0.0.0.0/0" }  
  ]  
}
```

EOF



macvlan 적용 확인하기

아래 명령어로 확인이 가능하다.

```
kubectl get net-attach-def -n default
```

```
kubectl get net-attach-def ext-macvlan-192-168-20 -n default -o yaml
```



테스트 Pod 배포 1

올바르게 구성이 되었는지 테스트 Pod를 구성해서 배포한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: v1  
kind: Pod  
metadata:  
  name: macvlan-test  
  namespace: default  
  annotations:  
    k8s.v1.cni.cncf.io/networks: ext-macvlan-192-168-20
```



테스트 Pod 배포 2

```
spec:
  containers:
  - name: bb
    image: busybox:1.36
    command: ["sh", "-c", "sleep 36000"]
EOF
```



NetworkAttachmentDefinition 생성

올바르게 구성이 되었는지 아래 명령어로 확인한다.

```
kubectl get pod macvlan-test -o wide  
kubectl describe pod macvlan-test | sed -n '/Annotations:/,/Events:/p'  
  
kubectl exec -it macvlan-test -- ip addr  
kubectl exec -it macvlan-test -- ip route
```



테스트 Pod 2 배포 1

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: macvlan-test2
  namespace: default
  annotations:
    k8s.v1.cni.cncf.io/networks: ext-macvlan-192-168-20
spec:
  containers:
```



테스트 Pod 2 배포 2

```
- name: bb
  image: busybox:1.36
  command: ["sh", "-c", "sleep 36000"]
  securityContext:
    capabilities:
      add: ["NET_RAW"]
```

EOF



최종 테스트

POD끼리 Ping를 서로 던져본다.

```
kubectl get pod -o wide | grep macvlan-test
kubectl exec -it macvlan-test -- ip addr sh net1
kubectl exec -it macvlan-test2 -- ip addr sh net1
```

```
kubectl exec -it macvlan-test -- ping -c 3 192.168.20.51
kubectl exec -it macvlan-test2 -- ping -c 3 192.168.20.50
PING 192.168.20.50 (192.168.20.50): 56 data bytes
64 bytes from 192.168.20.50: seq=0 ttl=64 time=0.272 ms
64 bytes from 192.168.20.50: seq=1 ttl=64 time=0.277 ms
64 bytes from 192.168.20.50: seq=2 ttl=64 time=0.101 ms
```



가상머신

리소스 모델



쿠버네티스에서 가상머신 자원

쿠버네티스 가상머신(Kubernetes Virtual Machine)은 VM을 쿠버네티스 리소스로 취급해서 Pod처럼 YAML로 생성·삭제·확장·관리하는 방식. 하지만, 자원 각기 따로 구별하기 때문에 Pod의 컨테이너와 Pod의 VM은 다르다.

1. VM = 쿠버네티스 CRD
2. 스케줄링 = kube-scheduler
3. 네트워크 = CNI
4. 스토리지 = PVC/CSI
5. 라이프사이클 = kubectl/API
6. 관리 명령어 = virtctl/API



가상머신 구조

쿠버네티스의 가상머신 구조는 다음과 같이 구성이 되어 있다.

- VM = VirtualMachine/VirtualMachineInstance
- 내부적으로는 libvirt + QEMU
- VM은 실제로 Pod 안에서 실행(이 부분은 컨테이너와 동일 하지만 런처가 별도로 존재)



가상머신 특징

쿠버네티스에서 가상머신은 다음과 같은 특징을 가지고 있다.

- VM을 YAML로 정의
- `kubectl get vm`, `kubectl start vm`
- CI/CD 파이프라인과 자연스럽게 연결(TKN/Jenkins 그리고 ArgoCD 활용 가능)
- GPU, SR-IOV, HugePage 지원



언제 사용하는게 좋을까?

쿠버네티스에서 가상머신 사용 조건은 다음과 같이 간단히 정리가 된다.

1. 컨테이너 + VM 혼합 워크로드
2. 기존 VM을 점진적으로 Kubernetes로 이전
3. DevOps/GitOps 환경
4. 서비스 용도 가상머신
5. 빠른 서비스 프로비저닝이 필요한 경우



엔터프라이즈 쿠버네티스 가상화

쿠버네티스 기반 가상머신 솔루션은 현재 다음과 같이 지원하고 있다.

1. 오픈시프트 가상화
2. 수세 가상화(이전: Harvester)
3. KubeSphere

바닐라 버전으로 설치 및 구성으로 엔터프라이즈 구성 및 운영은 가능하다. 다만, 아직 YAML 기반으로 Manifest를 지원하기 때문에 추후에 오퍼레이터 혹은 HelmChart 기반으로 구성 및 패키징이 필요하다.



혼동하면 안되는 쿠버네티스 가상화

쿠버네티스는 가상화/컨테이너화 중간 정도의 가상머신 기술이 있다. 이들은 완전한 가상머신 기술은 아니다.

기술	개념
Kata Containers	Pod = 경량 VM
Firecracker	초경량 microVM
gVisor	사용자 공간 커널



기존 가상머신과 쿠버네티스 가상머신 차이

다음과 같은 차이점이 있다.

구분	기존 가상화	Kubernetes VM
관리 단위	VM	CRD
API	벤더별	Kubernetes API
자동화	제한적	GitOps / CI/CD
네트워크	브리지/VLAN	CNI
확장성	클러스터 제한	수평 확장



라이프사이클 명령어

쿠버네티스에서 가상머신 관리 명령어는 매우 간단하다.

```
kubectl apply -f vm.yaml
```

```
kubectl start vm
```

```
kubectl stop vm
```

```
kubectl delete vm
```



가상머신 생성 조건

생성은 앞에서 미리 하였지만, YAML기반으로 구성해야 한다. 구성 시 다음과 같은 조건이 충족이 되어야지 가상머신 생성이 가능하다.

구분	주요 조건
Compute	vCPU, Memory, HugePage
Storage	PVC, 이미지 타입
Network	Pod Network: POD 네트워크 구성. 사용은 가능하나 고급 기능 사용이 불가능 Multus: 가상머신에 직접적으로 인터페이스 연결
Schedule	Affinity, 전용 노드
Lifecycle	Migration, HA
Security	RBAC, SELinux
OS	Cloud-init(필수는 아니지만 권장), 최소 한 개의 컨테이너로 감싼 이미지



이미지 업로드

이미지 업로드



이미지 설명

이미 앞에서 다루었지만, 가상머신 이미지를 생성 후 컨테이너 파일로 한번 감싸는 형태로 제공된다. 이미지가 준비가 완료되면 다음과 같은 단계로 이미지 업로드를 진행 및 수행한다.

VM Image → OCI Image → Upload/Import → PersistentVolume (PVC)

→ VirtualMachine → Pod (virt-launcher) → QEMU/KVM

이미지 업로드를 하기 위해서 최소 한 개의 레지스트리 혹은 HTTP(s) 웹 서버 저장소가 구성이 되어 있어야 한다. 아래 자원을 통해서 이미지를 클러스터에 등록한다.

1. CDI(Containerized Data Importer)
2. DataVolume(CRD)
3. PVC



HTTP/HTTPS 이미지 Import 1

웹 서버를 통해서 쿠버네티스 클러스터에 이미지를 배포한다.

```
cat <<'EOF' | kubectl -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: rocky9-image
spec:
  source:
    http:
      url: https://10.0.0.250/k8s-rocky9.qcow2
```



HTTP/HTTPS 이미지 Import 2

```
pvc:  
  accessModes:  
    - ReadWriteOnce  
  resources:  
    requests:  
      storage: 20Gi
```

EOF

```
kubectl get DataVolume
```



업로드 프록시 노출 1

업로드 전, NodePort로 외부에서 접근이 가능하도록 프록시 서비스를 열어둔다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  name: cdi-uploadproxy-nodeport
  namespace: cdi
spec:
  type: NodePort
  selector:
    cdi.kubevirt.io: cdi-uploadproxy
```



업로드 프록시 노출 2

ports:

- name: https
- port: 443
- targetPort: 8443
- nodePort: 32443
- protocol: TCP

EOF

```
kubectl get svc -n cdi
```



로컬 파일 업로드 (virtctl)

혹은 쿠버네티스 가상머신 관리 명령어로 업로드가 가능하다. Multus에서 올바르게 네트워크 구성이 안되어 있으면, 업로드가 잘 되지 않는다.

```
virtctl image-upload dv rocky9-image \
--size=20Gi \
--image-path=rocky9.qcow2 \
--uploadproxy-url=https://10.0.4.131:32443 \
--insecure \
--storage-class=nfs-client
kubectl get images
```



이미지 업로드 디버깅

```
kubectl -n cdi get pod -l cdi.kubevirt.io=cdi-uploadproxy -o wide  
kubectl -n cdi get pod -l cdi.kubevirt.io=cdi-uploadproxy -o  
jsonpath='{.items[0].metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-  
status}'; echo
```

```
curl -k https://172.16.236.145:443/healthz 2>/dev/null || true  
curl -k https://172.16.236.145:32443/ 2>/dev/null || true
```

```
kubectl -n cdi get pods | grep upload  
kubectl -n cdi describe pod cdi-uploadproxy-77df649b85-tlltt | sed -n  
'/Events:/,$p'
```

리턴값으로 동작 여부를 판단한다.

```
kubectl -n cdi get pods -o wide | grep -E 'upload|proxy'  
kubectl -n cdi logs deploy/cdi-uploadproxy --tail=50
```



ContainerDisk (컨테이너 이미지) 1

컨테이너 이미지로 감싼 경우에는 레지스트리 서버에 업로드 후, 사용이 가능하다. 이 경우에는 컨테이너에서 사용하는 가상머신 이미지는 PVC가 아닌 Overlay로 동작하기 때문에, OS영역에 정보는 가상머신 제거 시, 같이 제거가 된다.

```
podman tag rocky9-containerdisk:1.0 10.0.4.131:5000/kubevirt/rocky9-  
containerdisk:1.0  
podman push --tls-verify=false 10.0.4.131:5000/kubevirt/rocky9-  
containerdisk:1.0
```



ContainerDisk (컨테이너 이미지) 2

위와 같이 업로드 하였으면, 아래와 같이 디스크 선언이 가능하다. 프로덕트 용도로는 적절하지 않다.

disks:

- name: rootdisk

containerDisk:

image: 10.0.0.250:5000/kubevirt/rocky9-containerdisk:1.0



insecure는 /etc/registries.conf에서
설정을 추가적으로 해주어야 한다.



볼륨 기반(dv)

오픈스택 Cinder처럼 볼륨으로 구성을 원하는 경우, 아래와 같이 최소 한 개의 DV(PVC)를 선언해야 한다.

```
spec:
  domain:
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
  volumes:
    - name: rootdisk
      dataVolume:
        name: rocky9-image
```



볼륨 기반 이미지 업로드 확인(dv)

아래와 같이 출력이 되면 업로드가 완료.

```
kubectl get dv
```

NAME	PHASE	PROGRESS	RESTARTS	AGE
cirros-dv	Succeeded	N/A		6d6h
cirros-dv-test	UploadScheduled	N/A		6d3h
cirros-http-dv	Succeeded	100.0%		6d
cirros-latest-e2a45211b1f4	Succeeded	100.0%		5d11h

이와 같이 출력이 되면 올바르게 업로드가 안된 상태

1. ImportInProgress
2. Pending
3. Failed



정리 1

방식	구성 요소	장점	단점	사용 권장도
DataVolume (HTTP/HTTPS)	CDI+DataVolume +PVC	표준 방식, 자동화, 재현성 우수	이미지 서버 필요	매우 권장
DataVolume (virtctl upload)	CDI+DataVolume +PVC	폐쇄망/오프라인 가능	CLI 필요	권장
ContainerDisk	컨테이너 이미지	빠른 테스트	성능/영속성 부적합	사용가능
ISO 설치형	ISO+Empty PVC	전통적 방식	자동화 어려움, 비효율	가능, 추천하지 않음
HostPath/수동 PV C	PVC 직접 생성	단순	운영/보안 위험	비권장



정리 2

항목	DataVolume
표준성	KubeVirt 공식 방식
자동화	YAML/GitOps 완전 대응
상태 관리	Phase(Succeeded/Failed) 명확
검증	업로드 성공 여부 확인 가능
이식성	클러스터 간 이동 가능



정리 3

자주 오해하는 부분.

질문	현실
ContainerDisk가 표준이다	테스트용으로 많이 사용. 혹은 이미지 기반으로 배포가 필요한 경우 적절
ISO가 더 안정적이다	자동화 및 복제는 불가능
OpenStack 이미지 그대로 쓰면 된다	가능은 하나, 환경 전제 조금 다른 부분이 있음
DV는 선택 사항이다	사실상 권장 사항



가상머신 관리

생성/조회/상세/삭제



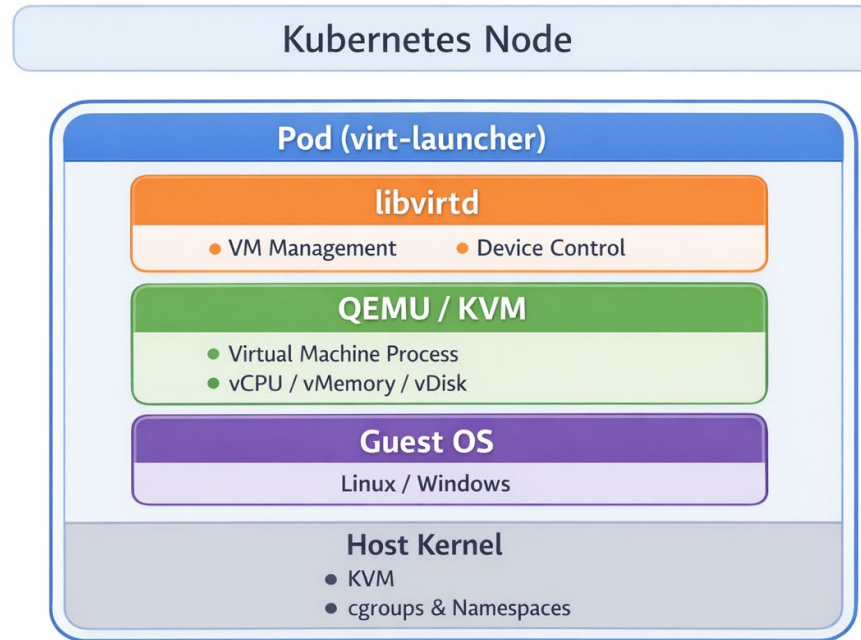
가상머신 생성 조건 정리

앞서 이야기 하였지만, 가상머신을 사용하기 위해서는 노드에 다음과 같은 조건이 충족이 되어야 한다.

1. Kubernetes 정상 동작
2. 컴퓨트 노드에 가상화 지원 CPU
 - Intel VT-x/AMD-V
 - SELinux 및 Firewalld
3. /dev/kvm 노드에 존재
4. Nested Virtualization 여부 확인(가상머신 속에서 다시 가상머신 구현하는 경우)



POD안 Libvirtd 구성



Virtual Machine Inside a Kubernetes Pod



테스트용 가상머신 생성 1

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cirros-01
  namespace: default
spec:
  running: true
  template:
    spec:
      domain:
        resources:
          requests:
            memory: 256Mi
```

가상머신 생성은 기본적으로
YAML 기반으로 생성이 가능하다.
명령어 생성은 기본적으로 불가능
하지만 명령어를 별도로 생성 후 가
능하다.

가상머신 상태 명시

현재 가상머신은 CPU/MEM/DISK
를 따로 구성한다.

계
속



테스트용 가상머신 생성 2

```
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
volumes:
- name: rootdisk
  persistentVolumeClaim:
    claimName: cirros-http-dv
```

사용할 디스크 백엔드 및 디스크 선언

EOF



기본 라이프사이클 명령어 적용

아래 명령어로 가상머신 중지/실행/제거를 해본다.

```
virtctl stop cirros-01  
kubectl get vmi  
virtctl start cirros-01  
kubectl get vmi  
kubectl delete vmi/cirros-01  
kubectl get vmi
```



템플릿 기반 가상머신 1

자원조합(Template)



자원조합(Template) 설명

쿠버네티스 가상화에는 다른 플랫폼에서 제공하는 템플릿 기능은 없다. 하지만, 동일 혹은 비슷하게 구성이 가능한 자원이 있다. 쿠버네티스 가상화에서 말하는 템플릿은 다음과 같은 방식을 통해서 구성이 된다.

VM Template=재사용 가능한 VM 생성 표준(자원/복제/풀)

1. CPU / Memory
2. Disk (기본 이미지)
3. Network
4. Cloud-init를 통해서 항상 같은 형태로 구성
5. 혹은 Virtual Machine ReplicaSet



자원조합 템플릿

자원 생성을 위해서 다음과 같은 순서로 자원을 생성한다.

Base Image

- DataVolume (read-only 이미지)

Clone DataVolume

- VM별 디스크 생성

공통 VM YAML

- CPU/Memory/Network

Cloud-init 변수화



Base Template Disk 가져오기 1

먼저 데이터 볼륨에 가상머신에서 사용할 디스크 이미지를 저장한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: rocky9-golden-image
spec:
  source:
    http:
      url: http://10.0.0.250/rocky9.qcow2
```



Base Template Disk 가져오기 2

```
pvc:  
  accessModes:  
    - ReadWriteOnce  
  resources:  
    requests:  
      storage: 20Gi
```

EOF

```
kubectl get dv -w
```

rocky9-golden-image	Succeeded	100.0%	1	5h12m
---------------------	-----------	--------	---	-------



플레이버 선언(InstanceType)

간단하게 아래와 같이 Flavor를 선언한다. 선언하지 않아도 사용이 가능하지만, 관리가 어렵다.

```
cat <<'EOF' | kubectl apply -f -  
kind: VirtualMachineClusterInstancetype  
metadata:  
  name: small  
spec:  
  cpu:  
    guest: 1  
  memory:  
    guest: 1Gi
```



OS 템플릿 (ClusterType)

혹은 미리 선언된 운영체제 타입을 사용하여도 된다.

```
kubectl get vmcp
```

```
linux          7d3h
```

```
linux.efi      7d3h
```

```
linux.virtiotransitional 7d3h
```

```
centos.stream9
```

```
kubectl describe vmcp/linux
```



가상머신 생성 1

인스턴스 생성은 아래와 같이 진행한다. 다만, 이 가상머신은 복제 용도로 사용하기 때문에 실행하지 않는다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: rocky9-from-template
spec:
  running: false
 instancetype:
    name: small
    kind: VirtualMachineClusterInstancetype
```



가상머신 생성 2

```
preference:
  name: linux
  kind: VirtualMachineClusterPreference
template:
  spec:
    domain:
      devices:
        disks:
          - name: rootdisk
            disk: { bus: virtio }
          - name: cloudinitdisk
            disk: { bus: virtio }
```



가상머신 생성 3

```
volumes:  
  - name: rootdisk  
    dataVolume:  
      name: rocky9-vm01-disk  
  - name: cloudinitdisk  
    cloudInitNoCloud:  
      userData: |  
        #cloud-config  
        hostname: rocky9-vm01
```



가상머신 생성 시 Disk Clone 1

가상머신을 생성한다. 이때 가상머신을 이전에 생성한 가상머신 이미지로 COW형태로 가상머신을 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: rocky9-vm01-disk
spec:
  source:
    pvc:
      name: rocky9-golden-image
      namespace: default
```



가상머신 생성 시 Disk Clone 2

두 번째 가상머신을 생성하려는 경우, 가상머신 이름을 변경 후 생성을 진행한다.

```
pvc:  
  accessModes:  
    - ReadWriteOnce  
  resources:  
    requests:  
      storage: 20Gi
```

EOF



자원기반 템플릿 가상머신 정리

위의 작업을 정리하면 다음과 같은 순서로 진행하였다.

[Template 역할 리소스]

- └─ DataVolume (Golden Image)
- └─ InstanceType
- └─ Preference

[VM YAML

- └─ name: vm01
- └─ rootdisk: dv-vm01
- └─ hostname: vm01

[VM YAML]

- └─ name: vm02
- └─ rootdisk: dv-vm02
- └─ hostname: vm02



템플릿 기반 가상머신 2

ReplicaSet/Clone 설명

Virtual Machine ReplicaSet



Replicas VS Pool 설명

실제로 가상머신 서비스를 구성하면 위 두 개의 서비스 기반으로 구성 및 구현한다. 어떠한 이유로 쿠버네티스 가상화에서 두 개의 자원으로 분리 및 운영 하는지 간단하게 알아본다.

조건

1. ReplicaSet은 VMI 개수 유지
2. Pool은 VM 묶음 관리

용도와 구현차이는 아래와 같다.

구분	VirtualMachineReplicaSet	VirtualMachinePool
관리 대상	VMI (VirtualMachineInstance)	VM (VirtualMachine)
성격	Stateless/교체 가능	Stateful/관리 대상
개념 비유	Pod ReplicaSet	VM 그룹
표준 사용성	제한적/특수	템플릿에 더 가까움



VirtualMachineReplicaSet(VMIRS)

항상 N개의 VMI가 동작해야 한다.

1. VMI가 죽으면 새 VMI를 즉시 생성
2. 디스크 상태 보존 개념 없음
3. 전통적인 VM 운영과는 거리 있음

주요 목적인 서비스 운영에 초점. 이러한 이유로 디스크는 컨테이너의 Pod처럼 PVC를 통해서 할당 후 데이터를 저장해야 한다.

다시 말하지만, VMI는 개수에 맞게 가상머신을 유지하는 게 주요 목적이다.



VirtualMachinePool(VMPool)

VM 여러 개를 하나의 Pool로 관리한다. 이 개념이 템플릿과 거의 흡사하다.

1. 각 VM은 고유한 이름/상태/디스크
2. start/stop/scale 가능
3. Template+Clone 패턴과 잘 맞음

디스크에 데이터 저장이 중요한 경우 반드시 VMPool 기반으로 구성한다.



디스크 관점에서 비교

항목	ReplicaSet	Pool
디스크	Ephemeral 중심	Persistent
PVC	공유/제약 큼	VM별 독립
Snapshot	부적합	적합
Backup	부적합	적합



운영자(서비스) 관점에서 비교

항목	ReplicaSet	Pool
VM 제어	개별 제어 어려움	개별 제어 가능
Scale	빠름	안정적
장애 복구	자동 교체	상태 유지
GitOps	제한적	매우 적합



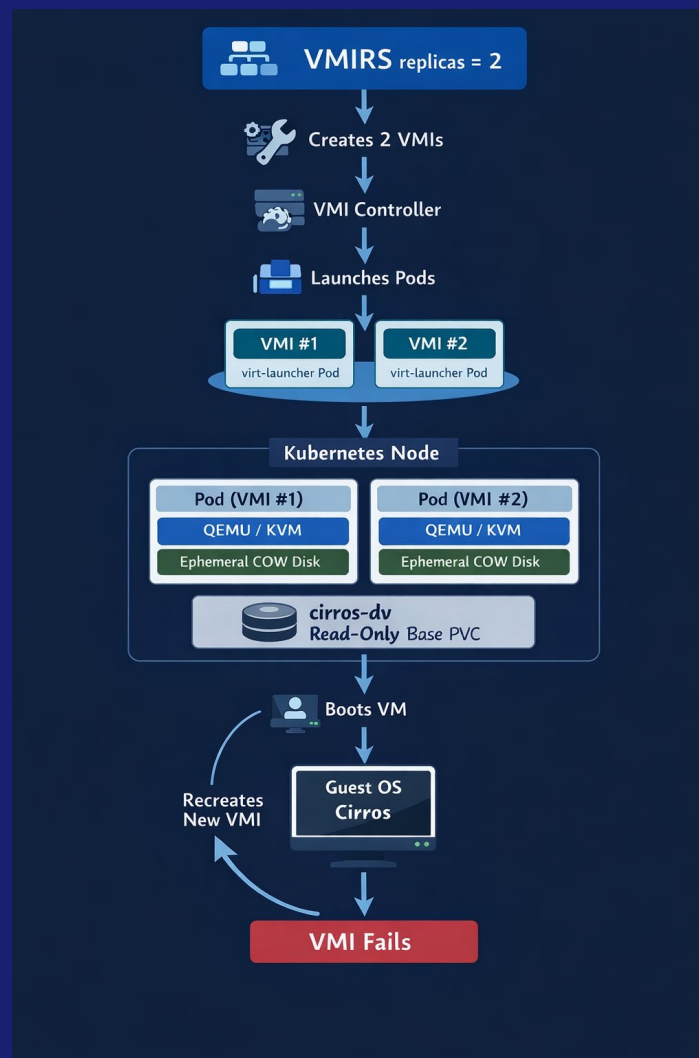
Virtual Machine ReplicaSet 구성

Virtual Machine ReplicaSet은 쿠버네티스에서 제공하는 ReplicaSet과 동일한 기능이다. 다만, 그 대상이 컨테이너에서 가상머신으로 변경이 된 부분이다.

이 자원의 주요 목적은 서비스를 위한 가상머신을 항상 해당 개수 만큼 유지하는 게 주요 목적이다. 기존에 구성된 vmf나 혹은 아래와 같이 사용자가 가상머신 사양을 직접 생성한다.



동작 방식 그림



인스턴스 타입 생성

```
cat <<EOF | kubectl apply -f -
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachineClusterInstancetype
metadata:
  name: vmf-small-1c-1g
spec:
  cpu:
    guest: 1
  memory:
    guest: 1Gi
EOF
kubectl get vmci
```



인스턴스 장치 타입 선언

```
cat <<EOF | kubectl apply -f -
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachineClusterPreference
metadata:
  name: pref-linux-virtio
spec:
  devices:
    preferredDiskBus: virtio
    preferredInterfaceModel: virtio
EOF
```



위의 내용으로 vmirs 구성 및 생성 1

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceReplicaSet
metadata:
  name: cirros-vmirs
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cirros-vmirs
```



위의 내용으로 vmirs 구성 및 생성 2

```
template:  
  metadata:  
    labels:  
      app: cirros-vmirs  
  spec:  
    domain:  
      resources:  
        requests:  
          memory: 256Mi
```



위의 내용으로 vmirs 구성 및 생성 3

```
devices:
  disks:
    - name: rootdisk
      disk:
        bus: virtio
  volumes:
    - name: rootdisk
      dataVolume:
        name: cirros-dv
```

EOF

```
kubectl get vmirs -w
```

```
kubectl get vmi -w
```

```
kubectl get pod
```



하지만!

위의 방식으로 하였을 때, 가상머신이 1개 이상이 생성이 되지 않는다. 그 이유는 다음과 같다.

- 현재 사용하는 디스크는 DV를 통해서 3개의 가상머신이 접근 시도

이러한 이유로, 한 개 이상의 프로세스가 블록 접근이 되지 않는다. 각 가상머신이 각각의 임시 디스크 기반으로 생성 및 구성이 되도록 한다. 이 구성은 오픈스택에서 Cinder가 아닌 Glance의 이미지 기반으로 오버레이 디스크 만드는 방법과 동일하다.

이러한 부분을 해결하기 위해서 다음처럼 코드를 수정한다.



디스크 부분 수정

```
volumes:  
  - ephemeral:  
      persistentVolumeClaim:  
        claimName: cirros-dv  
      name: rootdisk
```

EOF

```
kubectl get vmirs -w
```

```
kubectl get vmi -w
```

```
kubectl get pod
```

위와 같이 하면 각 가상머신 별로
디스크가 생성이 된다(COW)



템플릿 기반 가상머신 3

Virtual Machine Clone



VirtualMachinePool 설명 1

VirtualMachinePool은 여러 개의 VirtualMachine을 하나의 그룹으로 관리하기 위한 KubeVirt 리소스이다. Replica 수를 기준으로 VM을 생성·유지하며, 각 VM은 독립적인 상태와 디스크를 가진다.

Pool은 VM 템플릿을 기반으로 동일한 사양의 VM들을 생성하지만, 각 VM은 고유한 이름과 개별적인 라이프사이클을 가진다.

따라서 VM 단위로 시작, 정지, 삭제가 가능하며, 전통적인 가상머신 운영 방식과 가장 유사한 모델이다.

****여기서는 cirros 이미지 기반으로 구성한다.**



VirtualMachinePool 설명 2

스토리지 관점에서 VirtualMachinePool은 PersistentVolumeClaim(DataVolume clone) 기반 구성이 자연스럽다.

각 VM이 독립적인 디스크를 사용하므로 데이터가 유지되며, 재시작이나 장애 상황에서도 상태 보존이 가능하다.

이러한 특성으로 인해 VirtualMachinePool은 다음과 같은 워크로드에 적합하다.

1. 사용자별 VM 제공
2. 상태정보를 계속 유지하는 서비스
3. 클러스터 단위로 서비스 구성이 필요한 경우

반면, 단순히 개수만 유지하는 VMIRS와 달리, Pool은 운영과 관리 중심의 VM 그룹 관리를 목적으로 한다.



Pool 구성 및 생성 1

```
cat <<EOF | kubectl apply -f -
apiVersion: pool.kubevirt.io/v1alpha1
kind: VirtualMachinePool
metadata:
  name: cirros-pool
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cirros-pool
  virtualMachineTemplate:
    metadata:
      labels:
        app: cirros-pool
```



Pool 구성 및 생성 2

```
spec:
  running: false
 instancetype:
    name: vmf-small-1c-1g
    kind: VirtualMachineClusterInstancetype
  preference:
    name: pref-linux-virtio
    kind: VirtualMachineClusterPreference
```

앞에서 구성했던 내용 그대로 사용.



Pool 구성 및 생성 3

```
template:
  spec:
    domain:
      devices:
        disks:
          - name: rootdisk
            disk:
              bus: virtio
        networks:
          - name: default
            pod: {}
```

NAD(Multus)가 설치가 안되어
있으면 그냥 해당라인 삭제

계
속



Pool 구성 및 생성 4

```
volumes:  
  - name: rootdisk  
    dataVolume:  
      name: cirros-dv
```

EOF

```
kubectl get virtualmachinepools
```

NAME	DESIRED	CURRENT	READY	AGE
cirros-pool	2	2		59s



가상머신 스냅샷

가상머신 관리



KubeVirt 스냅샷 개념과 구조

KubeVirt에서 가상머신 스냅샷은 전통적인 하이퍼바이저 방식과 다르다. VM 자체를 멈추고 상태를 저장하는 것이 아니라, VM이 사용하는 스토리지(PVC)의 시점을 고정하는 방식이다.

KubeVirt의 가상머신은 VirtualMachine 리소스로 정의되며, 실제 디스크는 PVC(PersistentVolumeClaim) 를 통해 CSI 스토리지에 연결된다.

스냅샷 생성 시 KubeVirt는 VM과 PVC의 관계를 추적하여 해당 PVC에 대해 CSI VolumeSnapshot 생성을 요청한다.



KubeVirt 스냅샷 개념과 구조

이 구조에서 스냅샷의 실제 생성 주체는 KubeVirt가 아니라 **스토리지 드라이버(CSI)**이며, KubeVirt는 스냅샷 메타데이터와 VM 연관 관계만 관리한다.

이로 인해 KubeVirt 스냅샷은 다음과 같은 부분만 구성 및 관리한다

- 스토리지 네이티브
- 쿠버네티스 리소스 기반
- 선언적(YAML) 관리가 가능
- 메모리 부분에는 스냅샷에 포함하지 않음



스냅샷 복원과 설계 철학

KubeVirt 스냅샷은 메모리(RAM) 상태를 포함하지 않는다. 따라서 복원된 가상머신은 항상 Cold Start 상태로 시작한다. 이는 실시간 상태 복원은 불가능하지만, 대신 예측 가능하고 안정적인 복원을 보장한다.

스냅샷 복원은 VirtualMachineRestore 리소스를 통해 수행되며, 스냅샷 시점의 PVC 상태를 기준으로 디스크를 재구성한 뒤 VM에 다시 연결하는 방식으로 동작한다.

이러한 설계는 VM을 되돌리는 기능이 아니라 인프라 상태를 선언적으로 재현하는 방식에 가깝다.



PVC 볼륨 CRD 설치 및 생성

PVC 볼륨 스냅샷을 확인하기 위해서 아래와 같이 설치 및 진행한다. 설치가 되지 않는 경우, 생성 및 조회가 되지 않는다.

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v6.3.3/client/config/crd/snapshot.storage.k8s.io_volumesnapshots.yaml
```

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v6.3.3/client/config/crd/snapshot.storage.k8s.io_volumesnapshotcontents.yaml
```

```
kubectl get crd | grep volumesnapshot  
kubectl get volumesnapshotclass
```



스냅샷 생성 스토리지 클래스 생성

PVC 볼륨 스냅샷을 확인하기 위해서 아래와 같이 설치 및 진행한다. 설치가 되지 않는 경우, 생성 및 조회가 되지 않는다.

```
cat <<EOF | kubectl apply -f -
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: nfs-client
deletionPolicy: Delete
EOF
```



스냅샷 구성

먼저, 스냅샷을 구성할 가상머신이 있는지 확인한다.

```
kubectl get vm  
kubectl get vmi
```



스냅샷 구성

스냅샷을 구성하기 위해서 아래와 같이 진행한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: test-vm-snap-01
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
EOF
```



스냅샷 확인

올바르게 구성이 되었는지, 아래와 같이 확인이 가능하다.

```
kubectl get vmsnapshot
```

NAME	SOURCEKIND	SOURCENAME	PHASE	READYTOUSE	
cirros-snap	VirtualMachine	cirros-vm	Succeeded	true	7d18h



스냅샷 복원

복원은 다음 명령어로 가능하다. 특정 스냅샷을 test-vm-snap-01라는 이름으로 설정한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: snapshot.kubevirt.io/v1
kind: VirtualMachineRestore
metadata:
  name: test-vm-restore-01
spec:
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: test-vm
  virtualMachineSnapshotName: test-vm-snap-01
EOF
```



스냅샷 복원 확인

올바르게 복원이 되었는지 다음 명령어로 확인한다.

```
kubectl get vmrestore  
kubectl describe vmrestore test-vm-restore-01  
Complete: true
```



가상머신 마이그레이션



설명

쿠버네티스 기반 가상머신 라이브 마이그레이션이 가능하려면 몇 가지 필수 조건이 충족되어야 한다.

가장 중요한 요소는 **공유 스토리지**이다. 가상머신 디스크는 마이그레이션 전후 노드에서 동시에 접근 가능해야 하며, NFS나 Ceph과 같은 네트워크 기반 스토리지가 일반적으로 사용된다.

로컬 디스크 기반 가상머신은 구조적으로 라이브 마이그레이션이 불가능하다.

또한 CPU 호환성 역시 핵심 조건이다. 노드 간 CPU 기능 차이가 크면 마이그레이션이 실패할 수 있기 때문에, 공통 CPU 모델 기반으로 가상머신을 구성하는 것이 권장된다.

네트워크 측면에서는 기본 Pod 네트워크를 사용하는 경우 안정적인 마이그레이션이 가능하지만, SR-IOV나 macvlan과 같이 물리 NIC에 강하게 종속된 네트워크 구성에서는 제약이 발생한다.



설명

기존 OpenStack이나 VMware 환경과 비교하면, 쿠버네티스 가상머신 마이그레이션은 하이퍼바이저 중심이 아닌 **플랫폼 중심 제어**라는 점에서 큰 차이가 있다.

OpenStack에서는 Nova와 Hypervisor가 직접 마이그레이션을 수행하는 반면, 쿠버네티스에서는 마이그레이션 역시 하나의 워크로드 이벤트로 처리된다.

결과적으로 쿠버네티스 가상머신 마이그레이션은 **운영 자동화, 확장성, 정책 기반 제어에** 강점을 가지며, 클라우드 네이티브 환경에서 가상머신과 컨테이너를 함께 운영하기 위한 핵심 기능으로 자리 잡고 있다.



마이그레이션 조건

쿠버네티스 가상화에서는 마이그레이션을 위해서 다음과 같은 조건을 충족해야 한다.

1. 모든 노드가 가상머신을 실행할 수 있어야 한다
2. 공유 스토리지가 구성되어 있어야 한다
3. 노드 간 하드웨어(CPU)가 호환되어야 한다
4. 네트워크가 마이그레이션을 방해하지 않아야 한다
5. 라이브 마이그레이션 기능이 클러스터에서 활성화되어 있어야 한다



라이브 마이그레이션 기능 활성화

기본적으로 라이브 마이그레이션은 자동 활성화되지 않는다. 클러스터 레벨에서 명시적으로 기능을 켜야 한다.

```
kubectl -n kubevirt edit kubevirt kubevirt
```

```
spec:
```

```
  configuration:
```

```
    developerConfiguration:
```

```
      featureGates:
```

```
        - LiveMigration
```

```
kubectl -n kubevirt rollout restart deployment kubevirt-operator
```



라이브 마이그레이션 디스크

라이브 마이그레이션을 하려면 가상머신 디스크가 공유 스토리지 기반 PVC를 사용해야 한다.

- StorageClass
 - NFS, Ceph RBD
 - 여기서는 NFS 사용
- AccessMode
 - ReadWriteMany 또는 네트워크 블록 스토리지

아래와 같이 보통 PVC를 생성 및 구성한다.

```
volumes:  
- name: rootdisk  
  persistentVolumeClaim:  
    claimName: vm-root-pvc
```



라이브 마이그레이션 네트워크

라이브 마이그레이션 시 사용할 네트워크를 선언한다. 가급적인 분리 구성을 권장한다.

```
spec:  
  configuration:  
    migrationConfiguration:  
      network: migration-net
```



라이브 마이그레이션 CPU

라이브 마이그레이션 시 CPU오류를 방지하기 위해서 아래와 같이 설정한다.

```
cpu:  
  model: host-model
```



마이그레이션 수행

마이그레이션을 수행하기 위해서 아래와 같이 가상머신을 확인 후 진행한다.

```
kubectl get vmi | grep cirros-vm  
kubectl get vm | grep cirros-vm
```



마이그레이션 활성화 1

마이그레이션 기능이 활성화 되어 있는지 확인한다.

```
kubectl -n kubevirt get kubevirt kubevirt -o  
jsonpath='{.spec.configuration.developerConfiguration.featureGates}{"\n"}'
```

```
kubectl -n kubevirt patch kubevirt kubevirt --type merge -p \  
{  
"spec":{"configuration":{"developerConfiguration":{"featureGates":["LiveMig  
ration"]}}}  
}
```



마이그레이션 활성화 2

혹은 아래와 같이 패치/적용/확인이 가능하다.

```
kubectl -n kubevirt edit kubevirt kubevirt
```

```
spec:
```

```
  configuration:
```

```
    developerConfiguration:
```

```
      featureGates:
```

```
        - LiveMigration
```

```
kubectl -n kubevirt get kubevirt kubevirt -o
```

```
jsonpath='{.spec.configuration.developerConfiguration.featureGates}{"\n"}'
```



마이그레이션 실행

아래처럼 특정 가상머신을 명시 후, 마이그레이션을 수행한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceMigration
metadata:
  name: cirros-vm-mig-01
  namespace: default
spec:
  vmiName: cirros-vm
EOF
```



마이그레이션 실행

아래처럼 특정 가상머신을 명시 후, 마이그레이션을 수행한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceMigration
metadata:
  name: cirros-vm-mig-01
  namespace: default
spec:
  vmiName: cirros-vm
EOF
```



마이그레이션 확인

올바르게 마이그레이션이 되었는지 아래 명령어로 확인이 가능하다.

```
kubectl get vmim -n default
kubectl describe vmim -n default cirros-vm-mig-01
kubectl describe vmi -n default cirros-vm
kubectl get vmi -n default cirros-vm -o wide
```



특정 노드로 마이그레이션 선언

특정 노드로 마이그레이션을 위해서 아래와 같이 선언 후 실행한다.

```
kubectl label node node2 mig-target=true  
kubectl get node node2 --show-labels
```



특정 노드로 마이그레이션 실행

마이그레이션 파일을 아래와 같이 작성한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceMigration
metadata:
  name: cirros-vm-mig-01
  namespace: default
  annotations:
    kubevirt.io/migrationTargetNodeName: node2
spec:
  vmiName: cirros-vm
EOF
```



특정 노드로 마이그레이션 확인

올바르게 수행이 되었는지 아래 명령어로 확인한다.

```
kubectl get vmim -n default
```

```
kubectl describe vmim -n default cirros-vm-mig-01
```

NAME	PHASE	IP	NODE	
cirros-vm	Running	10.244.1.15	node1	← before
cirros-vm	Running	10.244.2.21	node2	← after



CI/CD

디스크 이미지 빌드 자동화

가상머신 기반 KNATIVE





모니터링

기존 쿠버네티스와 다른 부분
모니터링 시스템 구축





쿠버네티스 가상화 장/단점

Kube-Virt 한 줄

어떤 환경에 적합한가

Kubernetes 가상화의 미래



